

MODULE – 2

RELATIONAL MODEL

RELATIONAL ALGEBRA

MAPPING CONCEPTUAL DESIGN INTO A LOGICAL DESIGN

PREPARED BY MUSHTAQ AHMED D.M ASSISTANT PROFESSOR,
DEPT. OF ISE

Relational Model Concepts

- The relational Model of Data is based on the concept of a Relation.
- A Relation is a mathematical concept based on the ideas of sets.
- When a relation is thought of as a table of values, each row(tuple) in the table represents a collection of related data values.
- A row(tuple) represents a fact that typically corresponds to a real-world entity or relationship.
- The table name and column names are used to help to interpret the meaning of the values in each row.
- In the formal relational model terminology
 - (i) a row is called a tuple
 - (ii) a column header is called an attribute
 - (iii) the table is called a relation.

Informal Definitions

RELATION: A table of values

- A relation may be thought of as a **set of rows**.
- A relation may alternately be thought of as a **set of columns**.
- Each row represents a fact that corresponds to a real-world **entity** or **relationship**.
- Each row has a value of an item or set of items that uniquely identifies that row in the table.
- Sometimes row-ids or sequential numbers are assigned to identify the rows in the table.
- Each column typically is called by its column name or column header or attribute name.

Domains, Attributes, Tuples and Relations

Domain

- A **domain** D is a set of atomic values.
-
- Atomic means that each value in the domain is indivisible as far as the relational model is concerned.
 - Usa_phone_numbers. The set of ten-digit phone numbers valid in the United States.
 - Social_security_numbers. The set of valid nine-digit Social Security numbers. (This is a unique identifier assigned to each person in the United States for employment, tax, and benefits purposes.)
 - Employee_ages. Possible ages of employees in a company; each must be an integer value between 15 and 80.
 - A data type or format is also specified for each domain.
 - * For example, the data type for the domain Usa_phone_numbers can be declared as a character string of the form (ddd)ddd-dddd, where each d is a numeric (decimal) digit and the first three digits form a valid telephone area code
 - * The data type for Employee_ages is an integer number between 15 and 80.

Domains, Attributes, Tuples and Relations

- Relation Schema

- **Relation schema** R , denoted by $R (A_1, A_2, \dots, A_n)$ is made up of a relation name R and a list of attributes $A_1, A_2, \dots, A_n$
- Each **attribute** A_i is a name of a role played by some domain D in the relation schema R .
- D is called the domain of A_i and is denoted by **dom**(A_i).
- A relation schema is used to *describe* a relation; R is called the **name** of this relation

- Degree of the relation

The **degree** or **arity** of a relation is the number of attributes n of its relation schema.

- A relation of degree seven, which stores information about university students, would contain seven attributes describing each student as follows:

STUDENT(Name, Ssn, Home_phone, Address, Office_phone, Age, Gpa)

- Using the data type of each attribute, the definition is sometimes written as:

STUDENT(Name: string, Ssn: string, Home_phone: string, Address: string, Office_phone: string, Age: integer, Gpa: real)

- For this relation schema, STUDENT is the name of the relation, which has seven attributes.

dom(Name) = Names; dom(Ssn) = Social_security_numbers;

Contd...

Relation

- A **relation** or **relation state** r of the relation schema $R (A_1, A_2, \dots, A_n)$, also denoted by $r(R)$, is a set of n – tuples $r = \{t_1, t_2, \dots, t_m\}$.
- Each **n – tuple** t is an ordered list of n values $t = \langle v_1, v_2, \dots, v_n \rangle$, where each v_i , $1 \leq i \leq n$, is an element of $\text{dom}(A_i)$ or is a special NULL value.

The terms relation intension for the relation schema R and relation extension for a relation state $r(R)$ are commonly used.

Mathematical Definition

A relation schema or relation state $r(R)$ is a mathematical relation of degree n on the domains $\text{dom}(A_1), \text{dom}(A_2), \dots, \text{dom}(A_n)$ which is a subset of the Cartesian product of the domains that defines R :

$$r(R) \subseteq (\text{dom}(A_1) \times \text{dom}(A_2) \times \dots \times \text{dom}(A_n))$$

The Cartesian product specifies all possible combinations of values from the underlying domains.

The total number of values, or cardinality in a domain D by $|D|$, the total number of tuples in the Cartesian product is

$$|\text{dom}(A_1)| \times |\text{dom}(A_2)| \times \dots \times |\text{dom}(A_n)|$$

Current relation state

- A relation state at a given time is called the current relation state.

Example...

Relation name

Attributes

STUDENT	Name	SSN	HomePhone	Address	OfficePhone	Age	GPA
	Benjamin Bayer	305-61-2435	373-1616	2918 Bluebonnet Lane	null	19	3.21
	Katherine Ashly	381-62-1245	375-4409	125 Kirby Road	null	18	2.89
	Dick Davidson	422-11-2320	null	3452 Elgin Road	749-1253	25	3.53
	Charles Cooper	489-22-1100	376-9821	265 Lark Lane	749-6492	28	3.93
	Barbara Benson	533-69-1238	839-8461	7384 Fontana Lane	null	19	3.25

Tuples

Characteristics of Relations

1. Ordering of tuples in a relation
2. Ordering of values within a tuple and an Alternative definition of a relation
3. Values and NULLs in tuples
4. Interpretation (Meaning) of a relation

Ordering of Tuples in a Relation

A relation is a set of tuples.

- ❖ The tuples are not considered to be ordered, even though they appear to be in the tabular form.

Relation is not sensitive to the ordering of tuples.

Ordering of values within a tuple and an Alternative definition of a relation

From the definition of the relation, an n – tuple is a ordered list of values, so the ordering of values in a tuple is important.

Alternative Definition

- A relation schema $R = \{A_1, A_2, \dots, A_n\}$ is a set of attributes, and a relation state $r(R)$ is a finite set of mappings $r = \{t_1, t_2, \dots, t_m\}$, where each tuple t_i is a mapping from R to D , and D is the union of the attribute domains; that is, $D = \text{dom}(A_1) \cup \text{dom}(A_2) \cup \dots \cup \text{dom}(A_n)$.

Values and NULLs in the Tuples

All values are considered *atomic* (indivisible).

This model sometimes called as the flat relational model.

A special **NULL** value is used to represent values that are unknown or inapplicable to certain tuples.

Interpretation (Meaning) of a Relation

The relation schema can be interpreted as a declaration or a type of **assertion**.

For example, the schema of the student entity has a Name, SSN, HomePhone, Address, Officephone, Age and GPA. Each tuple in the relation can then be interpreted as a fact or a particular instance of the relation.

Example...

STUDENT	Name	SSN	HomePhone	Address	OfficePhone	Age	GPA
	Dick Davidson	422-11-2320	null	3452 Elgin Road	749-1253	25	3.53
	Barbara Benson	533-69-1238	839-8461	7384 Fontana Lane	null	19	3.25
	Charles Cooper	489-22-1100	376-9821	265 Lark Lane	749-6492	28	3.93
	Katherine Ashly	381-62-1245	375-4409	125 Kirby Road	null	18	2.89
	Benjamin Bayer	305-61-2435	373-1616	2918 Bluebonnet Lane	null	19	3.21

Relational Model Notations

Following notations are used in the Relational Model:

- A relation schema R of degree n is denoted by $R(A_1, A_2, \dots, A_n)$.
- The letters Q, R, S denote relation names.
- The letters q, r, s denote relation states.
- The letters t, u, v denote tuples.

Relational Model Constraints and Relational Database Schemas

Constraints on databases can generally be divided into three main categories:

- Inherent model based or Implicit constraint
 - Constraints that are inherent to the data model-the constraint that a relation cannot have duplicate tuples is an inherent constraint.
- Schema based or Explicit constraint
 - Constraints that are directly expressed in schemas of the data model
- Application based or Semantic constraint or Business rules
 - Constraints that can not be directly expressed in schemas of the data model and hence must be expressed and enforced by the application programs-Ex: Writing assertions in application side

Constraints in the Relational Model

Domain Constraints

Key Constraints and Constraint on NULL values

Entity integrity constraints

Referential integrity constraints

Domain Constraints

These constraints specify that within each tuple, the value of each attribute A must be an atomic value from the domain $\text{dom}(A)$.

The data types associated with domains typically include standard numeric data types for integers and real numbers.

Key Constraints and Constraints on NULL values

Super Key

- It is a subset of attributes SK where any two distinct tuples t_1 and t_2 in a relation state r of R , we have the constraint that $t_1[SK] \neq t_2[SK]$

Key

- A key K of a relation schema R is a super key of R with the additional property that removing any attribute A from K leaves set of attributes K' that is not a super key of R any more.

Candidate Key

- A relation schema may have more than one key. In this case, each of the keys is called a candidate key.

Primary Key

- This is the candidate key whose values are used to identify tuples in the relation.
- The primary key of the relation schema are underlined.

Relational Databases and Relational Database Schemas

Relational Database Schema

- A relational database schema S is a set of relation schemas $S = \{R_1, R_2, \dots, R_m\}$ and a set of integrity constraints IC .

Relational Database State

- A relational database state DB of S is a set of relation states $DB = \{r_1, r_2, \dots, r_m\}$ such that each r_i is a state of R_i and such that the r_i relation states satisfy the integrity constraints specified in IC .

A database state that does not obey all the integrity constraints is called an **invalid state**, and a state that satisfies all the constraints in IC is called a **valid state**.

Entity Integrity Constraints

The entity integrity constraint states that no primary key can be NULL.

This is because the primary key value is used to identify individual tuples in a relation.

Having NULL values for the primary key implies that we can not identify some tuples.

For example, if two or more tuples had NULL for their primary keys, we might not be able to distinguish them if we tried to reference them from other relations.

Schema Diagram

Referential integrity constraints displayed on the COMPANY relational database schema.

EMPLOYEE

Fname	Minit	Lname	<u>Ssn</u>	Bdate	Address	Sex	Salary	Super_ssn	Dno
-------	-------	-------	------------	-------	---------	-----	--------	-----------	-----

DEPARTMENT

Dname	<u>Dnumber</u>	Mgr_ssn	Mgr_start_date
-------	----------------	---------	----------------

DEPT_LOCATIONS

<u>Dnumber</u>	<u>Dlocation</u>
----------------	------------------

PROJECT

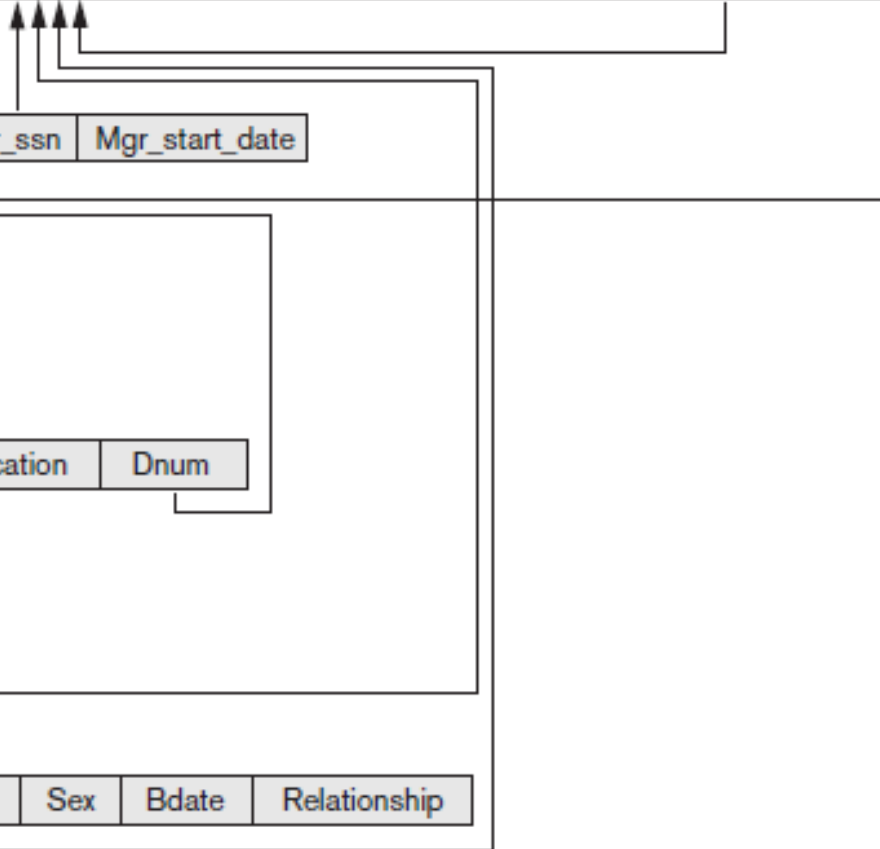
Pname	<u>Pnumber</u>	Plocation	Dnum
-------	----------------	-----------	------

WORKS_ON

<u>Essn</u>	<u>Pno</u>	Hours
-------------	------------	-------

DEPENDENT

<u>Essn</u>	<u>Dependent_name</u>	Sex	Bdate	Relationship
-------------	-----------------------	-----	-------	--------------



Example Database State

One possible database state for the COMPANY relational database schema.

EMPLOYEE

Fname	Minit	Lname	Ssn	Bdate	Address	Sex	Salary	Super_ssn	Dno
John	B	Smith	123456789	1965-01-09	731 Fondren, Houston, TX	M	30000	333445555	5
Franklin	T	Wong	333445555	1955-12-08	638 Voss, Houston, TX	M	40000	888665555	5
Alicia	J	Zelaya	999887777	1968-01-19	3321 Castle, Spring, TX	F	25000	987654321	4
Jennifer	S	Wallace	987654321	1941-06-20	291 Berry, Bellaire, TX	F	43000	888665555	4
Ramesh	K	Narayan	666884444	1962-09-15	975 Fire Oak, Humble, TX	M	38000	333445555	5
Joyce	A	English	453453453	1972-07-31	5631 Rice, Houston, TX	F	25000	333445555	5
Ahmad	V	Jabbar	987987987	1969-03-29	980 Dallas, Houston, TX	M	25000	987654321	4
James	E	Borg	888665555	1937-11-10	450 Stone, Houston, TX	M	55000	NULL	1

DEPARTMENT

Dname	Dnumber	Mgr_ssn	Mgr_start_date
Research	5	333445555	1988-05-22
Administration	4	987654321	1995-01-01
Headquarters	1	888665555	1981-06-19

DEPT_LOCATIONS

Dnumber	Dlocation
1	Houston
4	Stafford
5	Bellaire
5	Sugarland
5	Houston

WORKS_ON

Essn	Pno	Hours
123456789	1	32.5
123456789	2	7.5
666884444	3	40.0
453453453	1	20.0
453453453	2	20.0
333445555	2	10.0
333445555	3	10.0
333445555	10	10.0
333445555	20	10.0
999887777	30	30.0
999887777	10	10.0
987987987	10	35.0
987987987	30	5.0
987654321	30	20.0
987654321	20	15.0
888665555	20	NULL

PROJECT

Pname	Pnumber	Plocation	Dnum
ProductX	1	Bellaire	5
ProductY	2	Sugarland	5
ProductZ	3	Houston	5
Computerization	10	Stafford	4
Reorganization	20	Houston	1
Newbenefits	30	Stafford	4

DEPENDENT

Essn	Dependent_name	Sex	Bdate	Relationship
333445555	Alice	F	1986-04-05	Daughter
333445555	Theodore	M	1983-10-25	Son
333445555	Joy	F	1958-05-03	Spouse
987654321	Abner	M	1942-02-28	Spouse
123456789	Michael	M	1988-01-04	Son
123456789	Alice	F	1988-12-30	Daughter
123456789	Elizabeth	F	1967-05-05	Spouse

Referential Integrity Constraints

The referential integrity constraint is specified between two relations and is used to maintain the consistency among tuples in the two relations.

Informally, the referential integrity constraint states that a tuple in one relation that refers to another relation must refer to an existing tuple in that relation.

For example, the DNO of EMPLOYEE gives the department number for which each employee works; hence, its value in every EMPLOYEE tuple must match the DNUMBER value of some tuple in the DEPARTMENT relation.

Foreign Key

The condition for a foreign key specify a referential integrity constraint between the two relation schemas R_1 and R_2 .

A set of attributes FK in the relation schema R_1 is a foreign key of R_1 that references relation R_2 if it satisfies the following rules:

- The attributes in FK have the same domain(s) as the primary key attributes PK of R_2 ; the attributes FK are said to reference or refer to the relation R_2 .
- The value of FK in a tuple t_1 of the current state $r_1(R_1)$ either occurs as a value of PK for some tuple t_2 in the current state $r_2(R_2)$ or is NULL. In the former case, we have $t_1[\text{FK}] = t_2[\text{PK}]$, and we say that the tuple t_1 references or refers to the tuple t_2 .

In this definition, R_1 is called the referencing relation and R_2 is the referenced relation.'

Update Operations and Dealing with Constraint Violations

The operations of the relational model can be categorized into retrievals and update.

There are three basic update operations:

- Insert
- Delete
- Modify

The Insert Operation-All

The insert operation provides a list of attribute values for a new tuple t that is to be inserted into a relation R .

Insert can violate any of the four types of constraints.

1. Insert $\langle \text{'Cecilia'}, \text{'F'}, \text{'Kolonsky'}, \text{null}, \text{'1960-04-05'}, \text{'6357 Windy Lane, Katy, TX'}, \text{F}, 28000, \text{null}, 4 \rangle$ into EMPLOYEE.
2. Insert $\langle \text{'Alicia'}, \text{'J'}, \text{'Zelaya'}, \text{'999887777'}, \text{'1960-04-05'}, \text{'6357 Windy Lane, Katy, TX'}, \text{F}, 28000, \text{'987654321'}, 4 \rangle$ into EMPLOYEE.
3. Insert $\langle \text{'Cecilia'}, \text{'F'}, \text{'Kolonsky'}, \text{'677678989'}, \text{'1960-04-05'}, \text{'6357 Windswept, Katy, TX'}, \text{F}, 28000, \text{'987654321'}, 7 \rangle$ into EMPLOYEE.
4. Insert $\langle \text{'Cecilia'}, \text{'F'}, \text{'Kolonsky'}, \text{'677678989'}, \text{'1960-04-05'}, \text{'6357 Windy Lane, Katy, TX'}, \text{F}, 28000, \text{null}, 4 \rangle$ into EMPLOYEE.

The Delete Operation-RI

The delete operation is used to delete the tuples from the relation.

Eg:

Delete the WORKS_ON tuple with ESSN = '999887777' and PNO = 10.

Delete the EMPLOYEE tuple with SSN = '999887777'.

Delete the EMPLOYEE tuple with SSN = '333445555'.

The Update Operation

The update(or Modify) operation is used to change the values of one or more attributes in a tuple(or tuples) of some relation R.

Eg:

Update the SALARY of the EMPLOYEE tuple with SSN = '999887777' to 28000.

Update the DNO of the EMPLOYEE tuple with SSN = '999887777' to 1.

Update the DNO of the EMPLOYEE tuple with SSN = '999887777' to 7.

Update the SSN of the EMPLOYEE tuple with SSN = '999887777' to '987654321'.

The Transaction Concept

A database application program running against a relational database typically runs a series of transactions.

A transaction involves reading from the database as well as doing insertions, deletions and updates to existing values in the database.

These transactions must leave the database in a consistent state.

A single transaction may involve any number of retrieval operations that reads from the database and any number of update operations.

A large number of commercial applications running against relational databases in the Online Transaction Processing(OLTP) systems are executing transactions at rates several hundreds per second.

Relational Algebra Overview

- Relational algebra is the basic set of operations for the relational model
- These operations enable a user to specify **basic retrieval requests (or queries)**
- The result of an operation is a *new relation*, which may have been formed from one or more *input* relations
 - This property makes the algebra “closed” (all objects in relational algebra are relations)

The **algebra operations** thus produce new relations

- These can be further manipulated using operations of the same algebra

A sequence of relational algebra operations forms a **relational algebra expression**

- The result of a relational algebra expression is also a relation that represents the result of a database query (or retrieval request)

Relational Algebra Overview

Relational Algebra consists of several groups of operations

- **Unary Relational Operations**

- SELECT (symbol: σ (sigma))
- PROJECT (symbol: π (pi))
- RENAME (symbol: ρ (rho))

- **Relational Algebra Operations From Set Theory**

- UNION ($\cup$),
- INTERSECTION ($\cap$),
- DIFFERENCE (or MINUS, $-$)
- CARTESIAN PRODUCT ($\times$)

Relational Algebra Overview

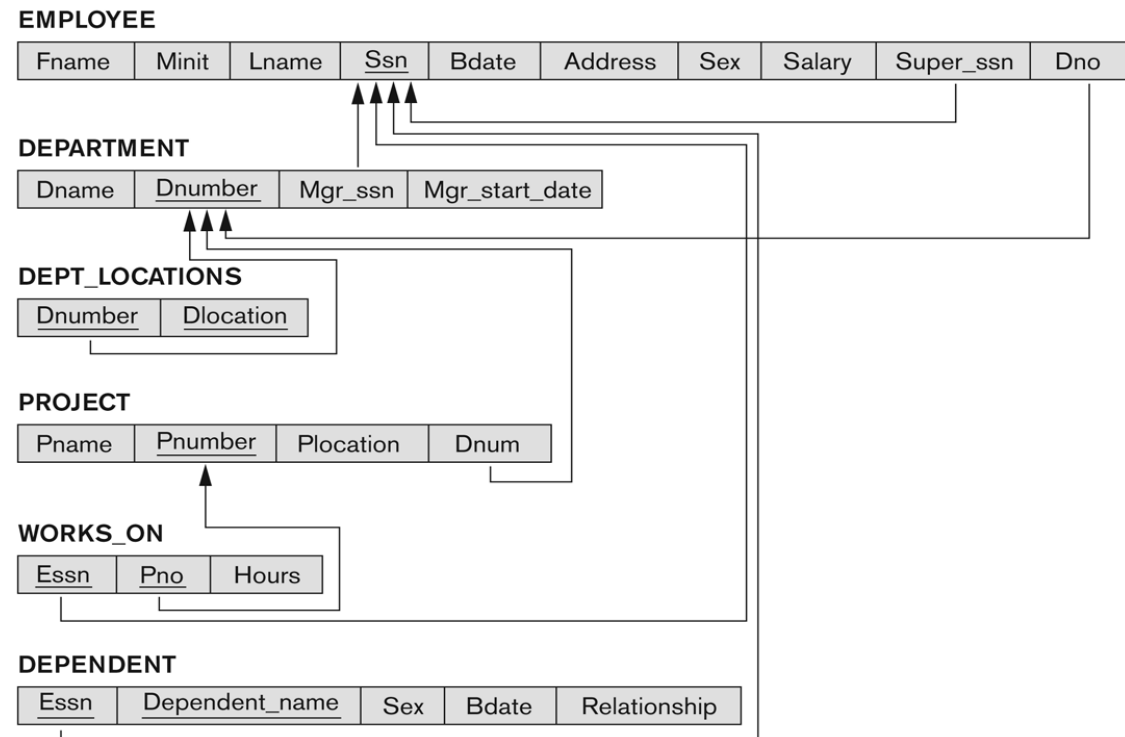
- **Binary Relational Operations**
 - JOIN (several variations of JOIN exist)
 - DIVISION
- **Additional Relational Operations**
 - OUTER JOINS, OUTER UNION
 - AGGREGATE FUNCTIONS
 - (These compute summary of information: for example, SUM, COUNT, AVG, MIN, MAX)

Database State for COMPANY

All examples discussed below refer to the COMPANY database shown here.

Figure 5.7

Referential integrity constraints displayed on the COMPANY relational database schema.



Unary Relational Operations: SELECT

The SELECT operation (denoted by σ (sigma)) is used to select a *subset* of the tuples from a relation based on a **selection condition**.

- The selection condition acts as a **filter**
- Keeps only those tuples that satisfy the qualifying condition
- Tuples satisfying the condition are *selected* whereas the other tuples are discarded (*filtered out*)

Examples:

- Select the EMPLOYEE tuples whose department number is 4:

$\sigma_{\text{DNUM} = 4} (\text{PROJECT})$

- Select the employee tuples whose salary is greater than \$30,000:

$\sigma_{\text{SALARY} > 30,000} (\text{EMPLOYEE})$

Unary Relational Operations: SELECT

- In general, the *select* operation is denoted by $\sigma_{\langle \text{selection condition} \rangle}(R)$ where
 - the symbol σ (sigma) is used to denote the *select* operator
 - the selection condition is a Boolean (conditional) expression specified on the attributes of relation R
 - tuples that make the condition **true** are selected
 - appear in the result of the operation
 - tuples that make the condition **false** are filtered out
 - discarded from the result of the operation

Unary Relational Operations: SELECT (contd.)

SELECT Operation Properties

- The SELECT operation $\sigma_{\langle \text{selection condition} \rangle}(R)$ produces a relation S that has the same schema (same attributes) as R
- SELECT σ is commutative:
 - $\sigma_{\langle \text{condition1} \rangle}(\sigma_{\langle \text{condition2} \rangle}(R)) = \sigma_{\langle \text{condition2} \rangle}(\sigma_{\langle \text{condition1} \rangle}(R))$
- Because of commutative property, a cascade (sequence) of SELECT operations may be applied in any order:
 - $\sigma_{\langle \text{cond1} \rangle}(\sigma_{\langle \text{cond2} \rangle}(\sigma_{\langle \text{cond3} \rangle}(R))) = \sigma_{\langle \text{cond2} \rangle}(\sigma_{\langle \text{cond3} \rangle}(\sigma_{\langle \text{cond1} \rangle}(R)))$
- A cascade of SELECT operations may be replaced by a single selection with a conjunction of all the conditions:
 - $\sigma_{\langle \text{cond1} \rangle}(\sigma_{\langle \text{cond2} \rangle}(\sigma_{\langle \text{cond3} \rangle}(R))) = \sigma_{\langle \text{cond1} \rangle \text{ AND } \langle \text{cond2} \rangle \text{ AND } \langle \text{cond3} \rangle}(R))$
- The number of tuples in the result of a SELECT is less than (or equal to) the number of tuples in the input relation R

The following query results refer to this database state

Figure 5.6

One possible database state for the COMPANY relational database schema.

EMPLOYEE

Fname	Minit	Lname	Ssn	Bdate	Address	Sex	Salary	Super_ssn	Dno
John	B	Smith	123456789	1965-01-09	731 Fondren, Houston, TX	M	30000	333445555	5
Franklin	T	Wong	333445555	1955-12-08	638 Voss, Houston, TX	M	40000	888665555	5
Alicia	J	Zelaya	999887777	1968-01-19	3321 Castle, Spring, TX	F	25000	987654321	4
Jennifer	S	Wallace	987654321	1941-06-20	291 Berry, Bellaire, TX	F	43000	888665555	4
Ramesh	K	Narayan	666884444	1962-09-15	975 Fire Oak, Humble, TX	M	38000	333445555	5
Joyce	A	English	453453453	1972-07-31	5631 Rice, Houston, TX	F	25000	333445555	5
Ahmad	V	Jabbar	987987987	1969-03-29	980 Dallas, Houston, TX	M	25000	987654321	4
James	E	Borg	888665555	1937-11-10	450 Stone, Houston, TX	M	55000	NULL	1

DEPARTMENT

Dname	Dnumber	Mgr_ssn	Mgr_start_date
Research	5	333445555	1988-05-22
Administration	4	987654321	1995-01-01
Headquarters	1	888665555	1981-06-19

DEPT_LOCATIONS

Dnumber	Dlocation
1	Houston
4	Stafford
5	Bellaire
5	Sugarland
5	Houston

WORKS_ON

Essn	Pno	Hours
123456789	1	32.5
123456789	2	7.5
666884444	3	40.0
453453453	1	20.0
453453453	2	20.0
333445555	2	10.0
333445555	3	10.0
333445555	10	10.0
333445555	20	10.0
999887777	30	30.0
999887777	10	10.0
987987987	10	35.0
987987987	30	5.0
987654321	30	20.0
987654321	20	15.0
888665555	20	NULL

PROJECT

Pname	Pnumber	Plocation	Dnum
ProductX	1	Bellaire	5
ProductY	2	Sugarland	5
ProductZ	3	Houston	5
Computerization	10	Stafford	4
Reorganization	20	Houston	1
Newbenefits	30	Stafford	4

DEPENDENT

Essn	Dependent_name	Sex	Bdate	Relationship
333445555	Alice	F	1986-04-05	Daughter
333445555	Theodore	M	1983-10-25	Son
333445555	Joy	F	1958-05-03	Spouse
987654321	Abner	M	1942-02-28	Spouse
123456789	Michael	M	1988-01-04	Son
123456789	Alice	F	1988-12-30	Daughter
123456789	Elizabeth	F	1967-05-05	Spouse

Unary Relational Operations: PROJECT

PROJECT Operation is denoted by π (pi)

This operation keeps certain *columns* (attributes) from a relation and discards the other columns.

- PROJECT creates a vertical partitioning
 - The list of specified columns (attributes) is kept in each tuple
 - The other attributes in each tuple are discarded

Example: To list each employee's first and last name and salary, the following is used:

$$\pi_{\text{LNAME, FNAME, SALARY}}(\text{EMPLOYEE})$$

Unary Relational Operations: PROJECT (cont.)

The general form of the *project* operation is:

$$\pi_{\langle \text{attribute list} \rangle}(R)$$

- π (pi) is the symbol used to represent the *project* operation
- $\langle \text{attribute list} \rangle$ is the desired list of attributes from relation R.

The project operation *removes any duplicate tuples*

- This is because the result of the *project* operation must be a *set of tuples*
 - Mathematical sets *do not allow* duplicate elements.

Unary Relational Operations: PROJECT (contd.)

PROJECT Operation Properties

- The number of tuples in the result of projection $\pi_{\langle \text{list} \rangle}(R)$ is always less or equal to the number of tuples in R
 - If the list of attributes includes a *key* of R , then the number of tuples in the result of PROJECT is *equal* to the number of tuples in R
- PROJECT is *not* commutative
 - $\pi_{\langle \text{list1} \rangle}(\pi_{\langle \text{list2} \rangle}(R)) = \pi_{\langle \text{list1} \rangle}(R)$ as long as $\langle \text{list2} \rangle$ contains the attributes in $\langle \text{list1} \rangle$

Examples of applying SELECT and PROJECT operations

Figure 6.1

Results of SELECT and PROJECT operations. (a) $\sigma_{(Dno=4 \text{ AND } Salary > 25000) \text{ OR } (Dno=5 \text{ AND } Salary > 30000)}(EMPLOYEE)$.
 (b) $\pi_{Lname, Fname, Salary}(EMPLOYEE)$. (c) $\pi_{Sex, Salary}(EMPLOYEE)$.

(a)

Fname	Minit	Lname	<u>Ssn</u>	Bdate	Address	Sex	Salary	Super_ssn	Dno
Franklin	T	Wong	333445555	1955-12-08	638 Voss, Houston, TX	M	40000	888665555	5
Jennifer	S	Wallace	987654321	1941-06-20	291 Berry, Bellaire, TX	F	43000	888665555	4
Ramesh	K	Narayan	666884444	1962-09-15	975 Fire Oak, Humble, TX	M	38000	333445555	5

(b)

Lname	Fname	Salary
Smith	John	30000
Wong	Franklin	40000
Zelaya	Alicia	25000
Wallace	Jennifer	43000
Narayan	Ramesh	38000
English	Joyce	25000
Jabbar	Ahmad	25000
Borg	James	55000

(c)

Sex	Salary
M	30000
M	40000
F	25000
F	43000
M	38000
M	25000
M	55000

Relational Algebra Expressions

We may want to apply several relational algebra operations one after the other

- Either we can write the operations as a single **relational algebra expression** by nesting the operations, or
- We can apply one operation at a time and create **intermediate result relations**.

In the latter case, we must give names to the relations that hold the intermediate results.

Single expression versus sequence of relational operations (Example)

To retrieve the first name, last name, and salary of all employees who work in department number 5, we must apply a select and a project operation

We can write a *single relational algebra expression* as follows:

- $\pi_{\text{FNAME, LNAME, SALARY}}(\sigma_{\text{DNO}=5}(\text{EMPLOYEE}))$

OR We can explicitly show the *sequence of operations*, giving a name to each intermediate relation:

- $\text{DEP5_EMPS} \leftarrow \sigma_{\text{DNO}=5}(\text{EMPLOYEE})$

- $\text{RESULT} \leftarrow \pi_{\text{FNAME, LNAME, SALARY}}(\text{DEP5_EMPS})$

Unary Relational Operations: RENAME

The RENAME operator is denoted by ρ (rho)

In some cases, we may want to *rename* the attributes of a relation or the relation name or both

- Useful when a query requires multiple operations
- Necessary in some cases (see JOIN operation later)

Unary Relational Operations: RENAME (contd.)

The general RENAME operation ρ can be expressed by any of the following forms:

- $\rho_S(B_1, B_2, \dots, B_n)(R)$ changes both:
 - the relation name to S , *and*
 - the column (attribute) names to $B_1, B_1, \dots, B_n$
- $\rho_S(R)$ changes:
 - the *relation name* only to S
- $\rho_{(B_1, B_2, \dots, B_n)}(R)$ changes:
 - the *column (attribute) names* only to $B_1, B_1, \dots, B_n$

Unary Relational Operations: RENAME (contd.)

For convenience, we also use a *shorthand* for renaming attributes in an intermediate relation:

- If we write:

- $\text{RESULT} \leftarrow \pi_{\text{FNAME, LNAME, SALARY}} (\text{DEP5_EMPS})$
- RESULT will have the *same attribute names* as DEP5_EMPS (same attributes as EMPLOYEE)

- If we write:

- $\text{RESULT} (\text{F, M, L, S, B, A, SX, SAL, SU, DNO}) \leftarrow \pi_{\text{FNAME, LNAME, SALARY}} (\text{DEP5_EMPS})$
- The 10 attributes of DEP5_EMPS are *renamed* to F, M, L, S, B, A, SX, SAL, SU, DNO, respectively

Example of applying multiple operations and RENAME

(a)

Fname	Lname	Salary
John	Smith	30000
Franklin	Wong	40000
Ramesh	Narayan	38000
Joyce	English	25000

(b)

TEMP

Fname	Minit	Lname	<u>Ssn</u>	Bdate	Address	Sex	Salary	Super_ssn	Dno
John	B	Smith	123456789	1965-01-09	731 Fondren, Houston,TX	M	30000	333445555	5
Franklin	T	Wong	333445555	1955-12-08	638 Voss, Houston,TX	M	40000	888665555	5
Ramesh	K	Narayan	666884444	1962-09-15	975 Fire Oak, Humble,TX	M	38000	333445555	5
Joyce	A	English	453453453	1972-07-31	5631 Rice, Houston, TX	F	25000	333445555	5

R

First_name	Last_name	Salary
John	Smith	30000
Franklin	Wong	40000
Ramesh	Narayan	38000
Joyce	English	25000

Figure 6.2

Results of a sequence of operations.

(a) $\pi_{\text{Fname, Lname, Salary}}(\sigma_{\text{Dno}=5}(\text{EMPLOYEE}))$.

(b) Using intermediate relations and renaming of attributes.

Relational Algebra Operations from Set Theory: UNION

UNION Operation

- Binary operation, denoted by $\cup$
- The result of $R \cup S$, is a relation that includes all tuples that are either in R or in S or in both R and S
- Duplicate tuples are eliminated
- The two operand relations R and S must be “type compatible” (or UNION compatible)
 - R and S must have same number of attributes
 - Each pair of corresponding attributes must be type compatible (have same or compatible domains)

Relational Algebra Operations from Set Theory: UNION

Example:

- To retrieve the social security numbers of all employees who either *work in department 5* (RESULT1 below) or *directly supervise an employee who works in department 5* (RESULT2 below)
- We can use the UNION operation as follows:

$$\text{DEP5_EMPS} \leftarrow \sigma_{\text{DNO}=5} (\text{EMPLOYEE})$$
$$\text{RESULT1} \leftarrow \pi_{\text{SSN}}(\text{DEP5_EMPS})$$
$$\text{RESULT2}(\text{SSN}) \leftarrow \pi_{\text{SUPERSSN}}(\text{DEP5_EMPS})$$
$$\text{RESULT} \leftarrow \text{RESULT1} \cup \text{RESULT2}$$

- The union operation produces the tuples that are in either RESULT1 or RESULT2 or both

Example of the result of a UNION operation

UNION Example

Figure 6.3

Result of the
UNION operation
 $\text{RESULT} \leftarrow \text{RESULT1} \cup \text{RESULT2}$.

RESULT1

Ssn
123456789
333445555
666884444
453453453

RESULT2

Ssn
333445555
888665555

RESULT

Ssn
123456789
333445555
666884444
453453453
888665555

Relational Algebra Operations from Set Theory

Type Compatibility of operands is required for the binary set operation UNION $\cup$, (also for INTERSECTION $\cap$, and SET DIFFERENCE $-$, see next slides)

$R1(A1, A2, \dots, An)$ and $R2(B1, B2, \dots, Bn)$ are type compatible if:

- they have the same number of attributes, and
- the domains of corresponding attributes are type compatible (i.e. $\text{dom}(Ai) = \text{dom}(Bi)$ for $i=1, 2, \dots, n$).

The resulting relation for $R1 \cup R2$ (also for $R1 \cap R2$, or $R1 - R2$, see next slides) has the same attribute names as the *first* operand relation $R1$ (by convention)

Relational Algebra Operations from Set Theory: INTERSECTION

INTERSECTION is denoted by $\cap$

The result of the operation $R \cap S$, is a relation that includes all tuples that are in both R and S

- The attribute names in the result will be the same as the attribute names in R

The two operand relations R and S must be “type compatible”

Relational Algebra Operations from Set Theory: SET DIFFERENCE (cont.)

SET DIFFERENCE (also called MINUS or EXCEPT) is denoted by –

The result of $R - S$, is a relation that includes all tuples that are in R but not in S

- The attribute names in the result will be the same as the attribute names in R

The two operand relations R and S must be “type compatible”

Example to illustrate the result of UNION, INTERSECT, and DIFFERENCE

(a) STUDENT

Fn	Ln
Susan	Yao
Ramesh	Shah
Johnny	Kohler
Barbara	Jones
Amy	Ford
Jimmy	Wang
Ernest	Gilbert

INSTRUCTOR

Fname	Lname
John	Smith
Ricardo	Browne
Susan	Yao
Francis	Johnson
Ramesh	Shah

(b)

Fn	Ln
Susan	Yao
Ramesh	Shah
Johnny	Kohler
Barbara	Jones
Amy	Ford
Jimmy	Wang
Ernest	Gilbert
John	Smith
Ricardo	Browne
Francis	Johnson

(c)

Fn	Ln
Susan	Yao
Ramesh	Shah

(d)

Fn	Ln
Johnny	Kohler
Barbara	Jones
Amy	Ford
Jimmy	Wang
Ernest	Gilbert

(e)

Fname	Lname
John	Smith
Ricardo	Browne
Francis	Johnson

Figure 6.4

The set operations UNION, INTERSECTION, and MINUS. (a) Two union-compatible relations. (b) $\text{STUDENT} \cup \text{INSTRUCTOR}$. (c) $\text{STUDENT} \cap \text{INSTRUCTOR}$. (d) $\text{STUDENT} - \text{INSTRUCTOR}$. (e) $\text{INSTRUCTOR} - \text{STUDENT}$.

Some properties of UNION, INTERSECT, and DIFFERENCE

Notice that both union and intersection are *commutative* operations; that is

- $R \cup S = S \cup R$, and $R \cap S = S \cap R$

Both union and intersection can be treated as n-ary operations applicable to any number of relations as both are *associative* operations; that is

- $R \cup (S \cup T) = (R \cup S) \cup T$
- $(R \cap S) \cap T = R \cap (S \cap T)$

The minus operation is not commutative; that is, in general

- $R - S \neq S - R$

Relational Algebra Operations from Set Theory: CARTESIAN PRODUCT

CARTESIAN (or CROSS) PRODUCT Operation

- This operation is used to combine tuples from two relations in a combinatorial fashion.
- Denoted by $R(A_1, A_2, \dots, A_n) \times S(B_1, B_2, \dots, B_m)$
- Result is a relation Q with degree $n + m$ attributes:
 - $Q(A_1, A_2, \dots, A_n, B_1, B_2, \dots, B_m)$, in that order.
- The resulting relation state has one tuple for each combination of tuples—one from R and one from S .
- Hence, if R has n_R tuples (denoted as $|R| = n_R$), and S has n_S tuples, then $R \times S$ will have $n_R * n_S$ tuples.
- The two operands do NOT have to be "type compatible"

Relational Algebra Operations from Set Theory: CARTESIAN PRODUCT (cont.)

Generally, CROSS PRODUCT is not a meaningful operation

- Can become meaningful when followed by other operations

Example (not meaningful):

- $\text{FEMALE_EMPS} \leftarrow \sigma_{\text{SEX}='F'}(\text{EMPLOYEE})$
- $\text{EMP_NAMES} \leftarrow \pi_{\text{FNAME, LNAME, SSN}}(\text{FEMALE_EMPS})$
- $\text{EMP_DEPENDENTS} \leftarrow \text{EMP_NAMES} \times \text{DEPENDENT}$

EMP_DEPENDENTS will contain every combination of EMP_NAMES and DEPENDENT

- whether or not they are actually related

Relational Algebra Operations from Set Theory: CARTESIAN PRODUCT (cont.)

To keep only combinations where the DEPENDENT is related to the EMPLOYEE, we add a SELECT operation as follows

Example (meaningful):

- $FEMALE_EMPS \leftarrow \sigma_{SEX='F'}(EMPLOYEE)$
- $EMP_NAMES \leftarrow \pi_{FNAME, LNAME, SSN}(FEMALE_EMPS)$
- $EMP_DEPENDENTS \leftarrow EMP_NAMES \times DEPENDENT$
- $ACTUAL_DEPS \leftarrow \sigma_{SSN=ESSN}(EMP_DEPENDENTS)$
- $RESULT \leftarrow \pi_{FNAME, LNAME, DEPENDENT_NAME}(ACTUAL_DEPS)$

RESULT will now contain the name of female employees and their dependents

Example of applying CARTESIAN PRODUCT

Figure 6.5

The CARTESIAN PRODUCT (CROSS PRODUCT) operation.

FEMALE_EMPS

Fname	Minit	Lname	Ssn	Bdate	Address	Sex	Salary	Super_ssn	Dno
Alicia	J	Zelaya	999887777	1968-07-19	3321 Castle, Spring, TX	F	25000	987654321	4
Jennifer	S	Wallace	987654321	1941-06-20	291 Berry, Bellaire, TX	F	43000	888665555	4
Joyce	A	English	453453453	1972-07-31	5631 Rice, Houston, TX	F	25000	333445555	5

EMPNames

Fname	Lname	Ssn
Alicia	Zelaya	999887777
Jennifer	Wallace	987654321
Joyce	English	453453453

EMP_DEPENDENTS

Fname	Lname	Ssn	Essn	Dependent_name	Sex	Bdate	...
Alicia	Zelaya	999887777	333445555	Alice	F	1986-04-05	...
Alicia	Zelaya	999887777	333445555	Theodore	M	1983-10-25	...
Alicia	Zelaya	999887777	333445555	Joy	F	1958-05-03	...
Alicia	Zelaya	999887777	987654321	Abner	M	1942-02-28	...
Alicia	Zelaya	999887777	123456789	Michael	M	1988-01-04	...
Alicia	Zelaya	999887777	123456789	Alice	F	1988-12-30	...
Alicia	Zelaya	999887777	123456789	Elizabeth	F	1967-05-05	...
Jennifer	Wallace	987654321	333445555	Alice	F	1986-04-05	...
Jennifer	Wallace	987654321	333445555	Theodore	M	1983-10-25	...
Jennifer	Wallace	987654321	333445555	Joy	F	1958-05-03	...
Jennifer	Wallace	987654321	987654321	Abner	M	1942-02-28	...
Jennifer	Wallace	987654321	123456789	Michael	M	1988-01-04	...
Jennifer	Wallace	987654321	123456789	Alice	F	1988-12-30	...
Jennifer	Wallace	987654321	123456789	Elizabeth	F	1967-05-05	...
Joyce	English	453453453	333445555	Alice	F	1986-04-05	...
Joyce	English	453453453	333445555	Theodore	M	1983-10-25	...
Joyce	English	453453453	333445555	Joy	F	1958-05-03	...
Joyce	English	453453453	987654321	Abner	M	1942-02-28	...
Joyce	English	453453453	123456789	Michael	M	1988-01-04	...
Joyce	English	453453453	123456789	Alice	F	1988-12-30	...
Joyce	English	453453453	123456789	Elizabeth	F	1967-05-05	...

ACTUAL_DEPENDENTS

Fname	Lname	Ssn	Essn	Dependent_name	Sex	Bdate	...
Jennifer	Wallace	987654321	987654321	Abner	M	1942-02-28	...

RESULT

Fname	Lname	Dependent_name
Jennifer	Wallace	Abner

Binary Relational Operations: JOIN

JOIN Operation (denoted by $\bowtie$)

- The sequence of CARTESIAN PRODECT followed by SELECT is used quite commonly to identify and select related tuples from two relations
- A special operation, called JOIN combines this sequence into a single operation
- This operation is very important for any relational database with more than a single relation, because it allows us *combine related tuples* from various relations
- The general form of a join operation on two relations $R(A_1, A_2, \dots, A_n)$ and $S(B_1, B_2, \dots, B_m)$ is:

$R \bowtie_{\langle \text{join condition} \rangle} S$

- where R and S can be any relations that result from general *relational algebra expressions*.

Binary Relational Operations: JOIN (cont.)

Example: Suppose that we want to retrieve the name of the manager of each department.

- To get the manager's name, we need to combine each DEPARTMENT tuple with the EMPLOYEE tuple whose SSN value matches the MGRSSN value in the department tuple.
- We do this by using the join $\bowtie$ operation.

- $\text{DEPT_MGR} \leftarrow \text{DEPARTMENT} \bowtie_{\text{MGRSSN=SSN}} \text{EMPLOYEE}$

MGRSSN=SSN is the join condition

- Combines each department record with the employee who manages the department
- The join condition can also be specified as $\text{DEPARTMENT.MGRSSN} = \text{EMPLOYEE.SSN}$

Example of applying the JOIN operation

DEPT_MGR

Dname	Dnumber	Mgr_ssn	...	Fname	Minit	Lname	Ssn	...
Research	5	333445555	...	Franklin	T	Wong	333445555	...
Administration	4	987654321	...	Jennifer	S	Wallace	987654321	...
Headquarters	1	888665555	...	James	E	Borg	888665555	...

Figure 6.6

Result of the JOIN operation

Some properties of JOIN

Consider the following JOIN operation:

- $R(A_1, A_2, \dots, A_n) \bowtie_{R.A_i=S.B_j} S(B_1, B_2, \dots, B_m)$
- Result is a relation Q with degree $n + m$ attributes:
 - $Q(A_1, A_2, \dots, A_n, B_1, B_2, \dots, B_m)$, in that order.
- The resulting relation state has one tuple for each combination of tuples— r from R and s from S , but *only if they satisfy the join condition* $r[A_i]=s[B_j]$
- Hence, if R has n_R tuples, and S has n_S tuples, then the join result will generally have *less than* $n_R * n_S$ tuples.
- Only related tuples (based on the join condition) will appear in the result

Some properties of JOIN

The general case of JOIN operation is called a Theta-join: $R \bowtie_{\theta} S$

The join condition is called *theta*

Theta can be any general boolean expression on the attributes of R and S; for example:

- $R.A_i < S.B_j \text{ AND } (R.A_k = S.B_l \text{ OR } R.A_p < S.B_q)$

Most join conditions involve one or more equality conditions “AND”ed together; for example:

- $R.A_i = S.B_j \text{ AND } R.A_k = S.B_l \text{ AND } R.A_p = S.B_q$

Binary Relational Operations: EQUIJOIN

EQUIJOIN Operation

The most common use of join involves join conditions with *equality comparisons* only

Such a join, where the only comparison operator used is =, is called an EQUIJOIN.

- In the result of an EQUIJOIN we always have one or more pairs of attributes (whose names need not be identical) that have identical values in every tuple.
- The JOIN seen in the previous example was an EQUIJOIN.

Binary Relational Operations:

NATURAL JOIN Operation

NATURAL JOIN Operation

- Another variation of JOIN called NATURAL JOIN — denoted by $*$ — was created to get rid of the second (superfluous) attribute in an EQUIJOIN condition.
- because one of each pair of attributes with identical values is superfluous
- The standard definition of natural join requires that the two join attributes, or each pair of corresponding join attributes, *have the same name* in both relations
- If this is not the case, a renaming operation is applied first.

Binary Relational Operations

NATURAL JOIN (contd.)

Example: To apply a natural join on the DNUMBER attributes of DEPARTMENT and DEPT_LOCATIONS, it is sufficient to write:

- $\text{DEPT_LOCS} \leftarrow \text{DEPARTMENT} * \text{DEPT_LOCATIONS}$

Only attribute with the same name is DNUMBER

An implicit join condition is created based on this attribute:

$\text{DEPARTMENT.DNUMBER} = \text{DEPT_LOCATIONS.DNUMBER}$

Another example: $Q \leftarrow R(A,B,C,D) * S(C,D,E)$

- The implicit join condition includes *each pair* of attributes with the same name, “AND”ed together:
 - $R.C = S.C$ AND $R.D = S.D$
- Result keeps only one attribute of each such pair:
 - $Q(A,B,C,D,E)$

Binary Relational Operations

NATURAL JOIN (contd.)

The NATURAL JOIN or EQUIJOIN operation can also be specified among multiple tables, leading to an *n-way join*

- Project-department-manager combined information
- This combines each project tuple with its controlling department tuple into a single Tuple
- Then combines that tuple with an employee tuple that is the department manager.
- Project_Dept

$((\text{PROJECT} \bowtie_{Dnum=Dnumber} \text{DEPARTMENT}) \bowtie_{Mgr_ssn=Ssn} \text{EMPLOYEE})$

Example of NATURAL JOIN operation

(a)

PROJ_DEPT

Pname	<u>Pnumber</u>	Plocation	Dnum	Dname	Mgr_ssn	Mgr_start_date
ProductX	1	Bellaire	5	Research	333445555	1988-05-22
ProductY	2	Sugarland	5	Research	333445555	1988-05-22
ProductZ	3	Houston	5	Research	333445555	1988-05-22
Computerization	10	Stafford	4	Administration	987654321	1995-01-01
Reorganization	20	Houston	1	Headquarters	888665555	1981-06-19
Newbenefits	30	Stafford	4	Administration	987654321	1995-01-01

(b)

DEPT_LOCS

Dname	Dnumber	Mgr_ssn	Mgr_start_date	Location
Headquarters	1	888665555	1981-06-19	Houston
Administration	4	987654321	1995-01-01	Stafford
Research	5	333445555	1988-05-22	Bellaire
Research	5	333445555	1988-05-22	Sugarland
Research	5	333445555	1988-05-22	Houston

Figure 6.7

Results of two NATURAL JOIN operations.

(a) PROJ_DEPT $\leftarrow$ PROJECT * DEPT.

(b) DEPT_LOCS $\leftarrow$ DEPARTMENT * DEPT_LOCATIONS.

Complete Set of Relational Operations

The set of operations including SELECT σ , PROJECT π , UNION $\cup$, DIFFERENCE $-$, RENAME ρ , and CARTESIAN PRODUCT $\times$ is called a *complete set* because any other relational algebra expression can be expressed by a combination of these five operations.

For example:

- $R \cap S = (R \cup S) - ((R - S) \cup (S - R))$
- $R \bowtie_{\langle \text{join condition} \rangle} S = \sigma_{\langle \text{join condition} \rangle} (R \times S)$

Binary Relational Operations: DIVISION

DIVISION Operation

- The division operation is applied to two relations
- $R(Z) \div S(X)$, where X subset Z . Let $Y = Z - X$ (and hence $Z = X \cup Y$); that is, let Y be the set of attributes of R that are not attributes of S .
- The result of DIVISION is a relation $T(Y)$ that includes a tuple t if tuples t_R appear in R with $t_R[Y] = t$, and with
 - $t_R[X] = t_s$ for every tuple t_s in S .



- For a tuple t to appear in the result T of the DIVISION, the values in t must appear in R in combination with *every* tuple in S .

Example of DIVISION

(a)

SSN_PNOS

Essn	Pno
123456789	1
123456789	2
666884444	3
453453453	1
453453453	2
333445555	2
333445555	3
333445555	10
333445555	20
999887777	30
999887777	10
987987987	10
987987987	30
987654321	30
987654321	20
888665555	20

SMITH_PNOS

Pno
1
2

SSNS

Ssn
123456789
453453453

(b)

R

A	B
a1	b1
a2	b1
a3	b1
a4	b1
a1	b2
a3	b2
a2	b3
a3	b3
a4	b3
a1	b4
a2	b4
a3	b4

S

A
a1
a2
a3

T

B
b1
b4

Figure 6.8

The DIVISION operation. (a) Dividing SSN_PNOS by SMITH_PNOS. (b) $T \leftarrow R \div S$.

Recap of Relational Algebra Operations

Table 6.1

Operations of Relational Algebra

Operation	Purpose	Notation
SELECT	Selects all tuples that satisfy the selection condition from a relation R .	$\sigma_{\langle \text{selection condition} \rangle}(R)$
PROJECT	Produces a new relation with only some of the attributes of R , and removes duplicate tuples.	$\pi_{\langle \text{attribute list} \rangle}(R)$
THETA JOIN	Produces all combinations of tuples from R_1 and R_2 that satisfy the join condition.	$R_1 \bowtie_{\langle \text{join condition} \rangle} R_2$
EQUIJOIN	Produces all the combinations of tuples from R_1 and R_2 that satisfy a join condition with only equality comparisons.	$R_1 \bowtie_{\langle \text{join condition} \rangle} R_2$, OR $R_1 \bowtie_{(\langle \text{join attributes 1} \rangle), (\langle \text{join attributes 2} \rangle)} R_2$
NATURAL JOIN	Same as EQUIJOIN except that the join attributes of R_2 are not included in the resulting relation; if the join attributes have the same names, they do not have to be specified at all.	$R_1 *_{\langle \text{join condition} \rangle} R_2$, OR $R_1 *_{(\langle \text{join attributes 1} \rangle), (\langle \text{join attributes 2} \rangle)} R_2$, OR $R_1 * R_2$
UNION	Produces a relation that includes all the tuples in R_1 or R_2 or both R_1 and R_2 ; R_1 and R_2 must be union compatible.	$R_1 \cup R_2$
INTERSECTION	Produces a relation that includes all the tuples in both R_1 and R_2 ; R_1 and R_2 must be union compatible.	$R_1 \cap R_2$
DIFFERENCE	Produces a relation that includes all the tuples in R_1 that are not in R_2 ; R_1 and R_2 must be union compatible.	$R_1 - R_2$
CARTESIAN PRODUCT	Produces a relation that has the attributes of R_1 and R_2 and includes as tuples all possible combinations of tuples from R_1 and R_2 .	$R_1 \times R_2$
DIVISION	Produces a relation $R(X)$ that includes all tuples $t[X]$ in $R_1(Z)$ that appear in R_1 in combination with every tuple from $R_2(Y)$, where $Z = X \cup Y$.	$R_1(Z) \div R_2(Y)$

Additional Relational Operations: Aggregate Functions and Grouping

A type of request that cannot be expressed in the basic relational algebra is to specify mathematical **aggregate functions** on collections of values from the database.

Examples of such functions include retrieving the average or total salary of all employees or the total number of employee tuples.

- These functions are used in simple statistical queries that summarize information from the database tuples.

Common functions applied to collections of numeric values include

- SUM, AVERAGE, MAXIMUM, and MINIMUM.

The COUNT function is used for counting tuples or values.

Aggregate Function Operation

Use of the Aggregate Functional operation $\mathcal{F}$

- $\mathcal{F}_{\text{MAX Salary}}(\text{EMPLOYEE})$ retrieves the maximum salary value from the EMPLOYEE relation
- $\mathcal{F}_{\text{MIN Salary}}(\text{EMPLOYEE})$ retrieves the minimum Salary value from the EMPLOYEE relation
- $\mathcal{F}_{\text{SUM Salary}}(\text{EMPLOYEE})$ retrieves the sum of the Salary from the EMPLOYEE relation
- $\mathcal{F}_{\text{COUNT SSN, AVERAGE Salary}}(\text{EMPLOYEE})$ computes the count (number) of employees and their average salary
 - Note: count just counts the number of rows, without removing duplicates

Using Grouping with Aggregation

The previous examples all summarized one or more attributes for a set of tuples

- Maximum Salary or Count (number of) Ssn

Grouping can be combined with Aggregate Functions

Example: For each department, retrieve the DNO, COUNT SSN, and AVERAGE SALARY

A variation of aggregate operation $\mathcal{F}$ allows this:

- Grouping attribute placed to left of symbol
- Aggregate functions to right of symbol
- $\text{DNO } \mathcal{F} \text{ COUNT SSN, AVERAGE Salary (EMPLOYEE)}$

Above operation groups employees by DNO (department number) and computes the count of employees and average salary per department

Examples of applying aggregate functions and grouping

Figure 6.10

The aggregate function operation.

(a) $\rho_{R(Dno, No_of_employees, Average_sal)} (Dno \bowtie COUNT Ssn, AVERAGE Salary (EMPLOYEE))$.

(b) $Dno \bowtie COUNT Ssn, AVERAGE Salary (EMPLOYEE)$.

(c) $\bowtie COUNT Ssn, AVERAGE Salary (EMPLOYEE)$.

R

(a)

Dno	No_of_employees	Average_sal
5	4	33250
4	3	31000
1	1	55000

(b)

Dno	Count_ssn	Average_salary
5	4	33250
4	3	31000
1	1	55000

(c)

Count_ssn	Average_salary
8	35125

Illustrating aggregate functions and grouping

Figure 8.6

Results of GROUP BY and HAVING. (a) Q24. (b) Q26.

(a)

Fname	Minit	Lname	<u>Ssn</u>	...	Salary	Super_ssn	Dno
John	B	Smith	123456789		30000	333445555	5
Franklin	T	Wong	333445555		40000	888665555	5
Ramesh	K	Narayan	666884444		38000	333445555	5
Joyce	A	English	453453453	...	25000	333445555	5
Alicia	J	Zelaya	999887777		25000	987654321	4
Jennifer	S	Wallace	987654321		43000	888665555	4
Ahmad	V	Jabbar	987987987		25000	987654321	4
James	E	Bong	888665555		55000	NULL	1

Grouping EMPLOYEE tuples by the value of Dno

Dno	Count (*)	Avg (Salary)
5	4	33250
4	3	31000
1	1	55000

Result of Q24

Additional Relational Operations (cont.)

Recursive Closure Operations

- Another type of operation that, in general, cannot be specified in the basic original relational algebra is **recursive closure**.
 - This operation is applied to a **recursive relationship**.
- An example of a recursive operation is to retrieve all SUPERVISEES of an EMPLOYEE e at all levels — that is, all EMPLOYEE e' directly supervised by e ; all employees e'' directly supervised by each employee e' ; all employees e''' directly supervised by each employee e'' ; and so on.

Additional Relational Operations (cont.)

Although it is possible to retrieve employees at each level and then take their union, we cannot, in general, specify a query such as “retrieve the supervisees of ‘James Borg’ at all levels” without utilizing a looping mechanism.

- The SQL3 standard includes syntax for recursive closure.

Additional Relational Operations (cont.)

(Borg's SSN is 888665555)

(SSN) (SUPERSSN)

SUPERVISION	SSN1	SSN2
	123456789	333445555
	333445555	888665555
	999887777	987654321
	987654321	888665555
	666884444	333445555
	453453453	333445555
	987987987	987654321

RESULT 1	SSN
	333445555
	987654321

(Supervised by Borg)

RESULT 2	SSN
	123456789
	999887777
	666884444
	453453453
	987987987

(Supervised by Borg's subordinates)

RESULT	SSN
	123456789
	999887777
	666884444
	453453453
	987987987
	333445555
	987654321

(RESULT1 $\cup$ RESULT2)

Additional Relational Operations (cont.) : OUTER JOIN

- The JOIN operations described earlier match tuples that satisfy the join condition. For example, for a NATURAL JOIN operation $R \bowtie S$, only tuples from R that have matching tuples in S —and vice versa—appear in the result
- Hence
 - (i) Tuples without a matching (or related) tuple are eliminated from the JOIN result.
 - (ii) Tuples with NULL values in the join attributes are also eliminated.
- This type of join, where tuples with no match are eliminated, is known as an inner join.
- This amounts to the loss of information if the user wants the result of the JOIN to include all the tuples in one or more of the component relations.

Additional Relational Operations (cont.)

The **left outer join** operation keeps every tuple in the first or left relation R in $R \bowtie S$; if no matching tuple is found in S, then the attributes of S in the join result are filled or “padded” with null values.

A similar operation, **right outer join**, keeps every tuple in the second or right relation S in the result of $R \bowtie S$.

A third operation, **full outer join**, denoted by $\bowtie$, keeps all tuples in both the left and the right relations when no matching tuples are found, padding them with null values as needed.

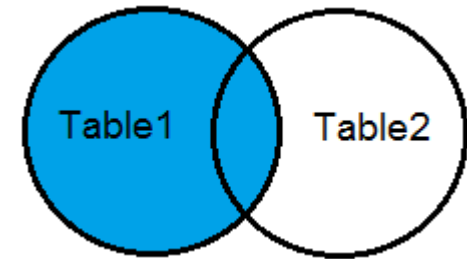
Additional Relational Operations (cont.)

```
TEMP ← (EMPLOYEE ⋈Ssn=Mgr_ssn DEPARTMENT)  
RESULT ←  $\pi_{Fname, Minit, Lname, Dname}(TEMP)$ 
```

RESULT

Fname	Minit	Lname	Dname
John	B	Smith	NULL
Franklin	T	Wong	Research
Alicia	J	Zelaya	NULL
Jennifer	S	Wallace	Administration
Ramesh	K	Narayan	NULL
Joyce	A	English	NULL
Ahmad	V	Jabbar	NULL
James	E	Borg	Headquarters

Figure 6.12
The result of a
LEFT OUTER JOIN
operation.



- The LEFT OUTER JOIN operation keeps every tuple in the *first*, or *left*, relation R in $R \bowtie S$;
- if no matching tuple is found in S , then the attributes of S in the join result are filled or *padded* with NULL values. The result of these operations is shown

Example Database State

One possible database state for the COMPANY relational database schema.

EMPLOYEE

Fname	Minit	Lname	Ssn	Bdate	Address	Sex	Salary	Super_ssn	Dno
John	B	Smith	123456789	1965-01-09	731 Fondren, Houston, TX	M	30000	333445555	5
Franklin	T	Wong	333445555	1955-12-08	638 Voss, Houston, TX	M	40000	888665555	5
Alicia	J	Zelaya	999887777	1968-01-19	3321 Castle, Spring, TX	F	25000	987654321	4
Jennifer	S	Wallace	987654321	1941-06-20	291 Berry, Bellaire, TX	F	43000	888665555	4
Ramesh	K	Narayan	666884444	1962-09-15	975 Fire Oak, Humble, TX	M	38000	333445555	5
Joyce	A	English	453453453	1972-07-31	5631 Rice, Houston, TX	F	25000	333445555	5
Ahmad	V	Jabbar	987987987	1969-03-29	980 Dallas, Houston, TX	M	25000	987654321	4
James	E	Borg	888665555	1937-11-10	450 Stone, Houston, TX	M	55000	NULL	1

DEPARTMENT

Dname	Dnumber	Mgr_ssn	Mgr_start_date
Research	5	333445555	1988-05-22
Administration	4	987654321	1995-01-01
Headquarters	1	888665555	1981-06-19

DEPT_LOCATIONS

Dnumber	Dlocation
1	Houston
4	Stafford
5	Bellaire
5	Sugarland
5	Houston

WORKS_ON

Essn	Pno	Hours
123456789	1	32.5
123456789	2	7.5
666884444	3	40.0
453453453	1	20.0
453453453	2	20.0
333445555	2	10.0
333445555	3	10.0
333445555	10	10.0
333445555	20	10.0
999887777	30	30.0
999887777	10	10.0
987987987	10	35.0
987987987	30	5.0
987654321	30	20.0
987654321	20	15.0
888665555	20	NULL

PROJECT

Pname	Pnumber	Plocation	Dnum
ProductX	1	Bellaire	5
ProductY	2	Sugarland	5
ProductZ	3	Houston	5
Computerization	10	Stafford	4
Reorganization	20	Houston	1
Newbenefits	30	Stafford	4

DEPENDENT

Essn	Dependent_name	Sex	Bdate	Relationship
333445555	Alice	F	1986-04-05	Daughter
333445555	Theodore	M	1983-10-25	Son
333445555	Joy	F	1958-05-03	Spouse
987654321	Abner	M	1942-02-28	Spouse
123456789	Michael	M	1988-01-04	Son
123456789	Alice	F	1988-12-30	Daughter
123456789	Elizabeth	F	1967-05-05	Spouse

Additional Relational Operations (cont.)

OUTER UNION Operations

- The outer union operation was developed to take the union of tuples from two relations if the relations are *not type compatible*.
- This operation will take the union of tuples in two relations $R(X, Y)$ and $S(X, Z)$ that are **partially compatible**, meaning that only some of their attributes, say X , are type compatible.
- The attributes that are type compatible are represented only once in the result, and those attributes that are not type compatible from either relation are also kept in the result relation $T(X, Y, Z)$.

Additional Relational Operations (cont.)

Example: An outer union can be applied to two relations whose schemas are STUDENT(Name, SSN, Department, Advisor) and INSTRUCTOR(Name, SSN, Department, Rank).

- Tuples from the two relations are matched based on having the same combination of values of the shared attributes— Name, SSN, Department.
- If a student is also an instructor, both Advisor and Rank will have a value; otherwise, one of these two attributes will be null.
- The result relation STUDENT_OR_INSTRUCTOR will have the following attributes:

STUDENT_OR_INSTRUCTOR (Name, SSN, Department, Advisor, Rank)

Query 1. Retrieve the name and address of all employees who work for the 'Research' department.

Query 2. For every project located in 'Stafford', list the project number, the controlling department number, and the department manager's last name, address, and birth date.

Examples of Queries in Relational Algebra

- **Q1: Retrieve the name and address of all employees who work for the 'Research' department.**

$\text{RESEARCH_DEPT} \leftarrow \sigma_{\text{DNAME}='Research'}(\text{DEPARTMENT})$

$\text{RESEARCH_EMPS} \leftarrow (\text{RESEARCH_DEPT} \bowtie_{\text{DNUMBER}=\text{DNOEMPLOYEE}} \text{EMPLOYEE})$

$\text{RESULT} \leftarrow \pi_{\text{FNAME}, \text{LNAME}, \text{ADDRESS}}(\text{RESEARCH_EMPS})$

- **Q6: Retrieve the names of employees who have no dependents.**

$\text{ALL_EMPS} \leftarrow \pi_{\text{SSN}}(\text{EMPLOYEE})$

$\text{EMPS_WITH_DEPS}(\text{SSN}) \leftarrow \pi_{\text{ESSN}}(\text{DEPENDENT})$

$\text{EMPS_WITHOUT_DEPS} \leftarrow (\text{ALL_EMPS} - \text{EMPS_WITH_DEPS})$

$\text{RESULT} \leftarrow \pi_{\text{LNAME}, \text{FNAME}}(\text{EMPS_WITHOUT_DEPS} * \text{EMPLOYEE})$

Example Database State

One possible database state for the COMPANY relational database schema.

EMPLOYEE

Fname	Minit	Lname	Ssn	Bdate	Address	Sex	Salary	Super_ssn	Dno
John	B	Smith	123456789	1965-01-09	731 Fondren, Houston, TX	M	30000	333445555	5
Franklin	T	Wong	333445555	1955-12-08	638 Voss, Houston, TX	M	40000	888665555	5
Alicia	J	Zelaya	999887777	1968-01-19	3321 Castle, Spring, TX	F	25000	987654321	4
Jennifer	S	Wallace	987654321	1941-06-20	291 Berry, Bellaire, TX	F	43000	888665555	4
Ramesh	K	Narayan	666884444	1962-09-15	975 Fire Oak, Humble, TX	M	38000	333445555	5
Joyce	A	English	453453453	1972-07-31	5631 Rice, Houston, TX	F	25000	333445555	5
Ahmad	V	Jabbar	987987987	1969-03-29	980 Dallas, Houston, TX	M	25000	987654321	4
James	E	Borg	888665555	1937-11-10	450 Stone, Houston, TX	M	55000	NULL	1

DEPARTMENT

Dname	Dnumber	Mgr_ssn	Mgr_start_date
Research	5	333445555	1988-05-22
Administration	4	987654321	1995-01-01
Headquarters	1	888665555	1981-06-19

DEPT_LOCATIONS

Dnumber	Dlocation
1	Houston
4	Stafford
5	Bellaire
5	Sugarland
5	Houston

WORKS_ON

Essn	Pno	Hours
123456789	1	32.5
123456789	2	7.5
666884444	3	40.0
453453453	1	20.0
453453453	2	20.0
333445555	2	10.0
333445555	3	10.0
333445555	10	10.0
333445555	20	10.0
999887777	30	30.0
999887777	10	10.0
987987987	10	35.0
987987987	30	5.0
987654321	30	20.0
987654321	20	15.0
888665555	20	NULL

PROJECT

Pname	Pnumber	Plocation	Dnum
ProductX	1	Bellaire	5
ProductY	2	Sugarland	5
ProductZ	3	Houston	5
Computerization	10	Stafford	4
Reorganization	20	Houston	1
Newbenefits	30	Stafford	4

DEPENDENT

Essn	Dependent_name	Sex	Bdate	Relationship
333445555	Alice	F	1986-04-05	Daughter
333445555	Theodore	M	1983-10-25	Son
333445555	Joy	F	1958-05-03	Spouse
987654321	Abner	M	1942-02-28	Spouse
123456789	Michael	M	1988-01-04	Son
123456789	Alice	F	1988-12-30	Daughter
123456789	Elizabeth	F	1967-05-05	Spouse

Relational Database Design Using ER – to – Relational Mapping

1. Mapping of Regular Entity Types
2. Mapping of Weak Entity Types
3. Mapping of Binary 1:1 Relationship Types
4. Mapping of Binary 1:N Relationship Types
5. Mapping of Binary M:N Relationship Types
6. Mapping of Multivalued Attributes
7. Mapping of N – ary Relationship Types

Mapping of Regular Entity Types

For each regular(strong) entity type E in the ER schema, create a relation R that includes all simple attributes of E.

Include only the simple component attributes of a composite attribute.

Choose one of the key attributes of E as the primary key of R.

If the chosen key of E is a composite, then the set of simple attributes that form it will together form the primary key of R.

If multiple keys were identified for E during the conceptual design, the information describing the attributes that form each additional key is kept in order to specify secondary keys of relation R.

Mapping of Weak Entity Types

For each weak entity type W in the ER schema with owner entity type E , create a relation R and include all simple attributes of W as attributes of R .

In addition, include as foreign key attributes of R , the primary key attribute(s) of the relation(s) that correspond to the owner entity type(s).

The primary key of R is the combination of the primary key(s) of the owner(s) and the partial key of the weak entity type W , if any.

If there is a weak entity type E_2 whose owner is also a weak entity type E_1 , then E_1 should be mapped before E_2 to determine its primary key first.

Mapping of Binary 1:1 Relationship Types

For each binary 1:1 relationship type R in ER schema, identify the relations S and T that correspond to the entity types participating in R .

There are three possible approaches:

- The foreign key approach
- The merged relationship approach
- The cross reference or relationship relation approach

Mapping of Binary 1:N Relationship Types

For each regular binary 1:N relationship type R, identify the relation S that represents the participating entity type at the N – side of the relationship type.

Include as foreign key in S the primary key of relation T that represents the other entity type participating in R.

Include any simple attributes of the 1:N relationship type as attributes of S.

Mapping of Binary M:N Relationship Types

For each binary M:N relationship type R, create a new relation S to represent R.

Include as foreign key attributes in S the primary keys of the relations that represent the participating entity types.

Their combination will form the primary key of S.

Also include any simple attributes of the M:N relationship type as attributes of S.

Mapping of Multivalued Attributes

For each Multivalued attribute A, create a new relation R.

This relation R will include an attribute corresponding to A, plus the primary key attribute K – as foreign key in R – of the relation that represents the entity type or the relationship type that has A as an attribute.

The primary key of R is the combination of A and K.

Mapping of N – ary Relationship Types

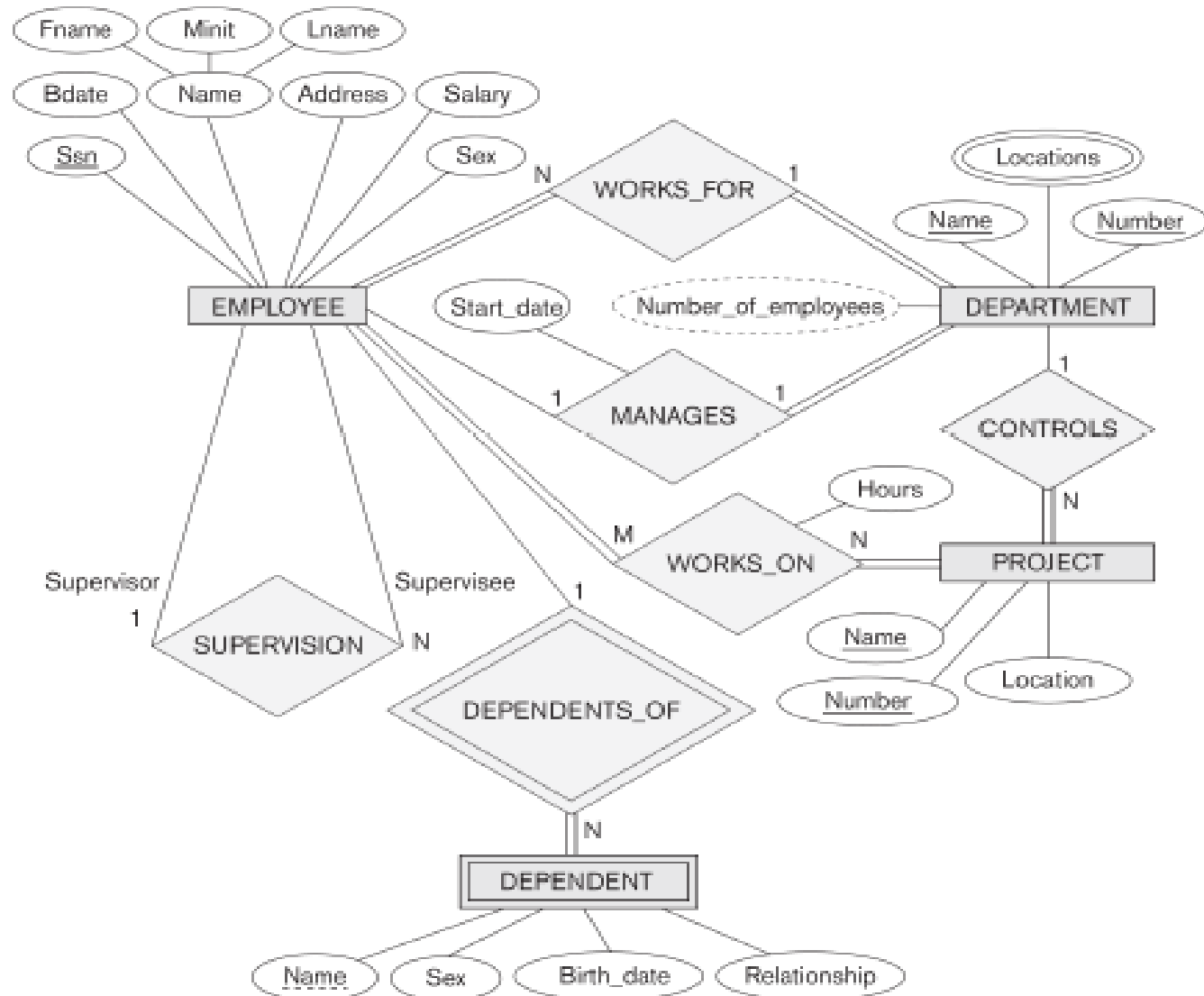
For each N – ary relationship type R, where $n > 2$, create a new relation S to represent R.

Include as foreign key attributes in S the primary keys of the relations that represent the participating entity types.

Also include any simple attributes of the n – ary relationship type as attributes of S.

The primary key of S is usually a combination of all the foreign keys that reference the relations representing the participating entity types.

Example ERD



Resulting Relational Schema

EMPLOYEE

Fname	Minit	Lname	<u>Ssn</u>	Bdate	Address	Sex	Salary	Super_ssn	Dno
-------	-------	-------	------------	-------	---------	-----	--------	-----------	-----

DEPARTMENT

Dname	<u>Dnumber</u>	Mgr_ssn	Mgr_start_date
-------	----------------	---------	----------------

DEPT_LOCATIONS

<u>Dnumber</u>	<u>Dlocation</u>
----------------	------------------

PROJECT

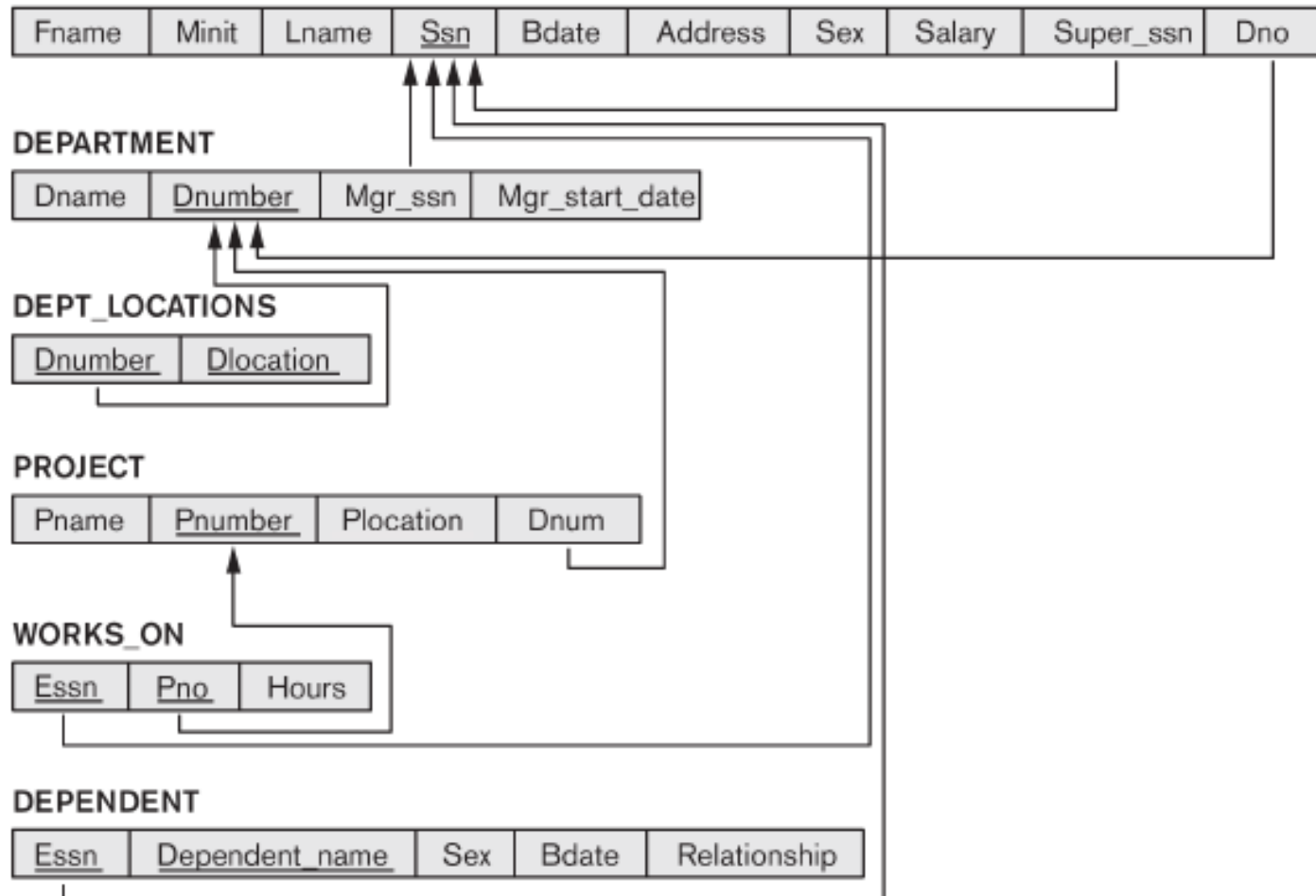
Pname	<u>Pnumber</u>	Plocation	Dnum
-------	----------------	-----------	------

WORKS_ON

<u>Essn</u>	<u>Pno</u>	Hours
-------------	------------	-------

DEPENDENT

<u>Essn</u>	<u>Dependent_name</u>	Sex	Bdate	Relationship
-------------	-----------------------	-----	-------	--------------



Step 1: Regular Entity Types

- i. For each regular/strong entity type, create a corresponding relation that includes all the simple attributes (includes simple attributes of composite relations)
- ii. Choose one of the key attributes as primary
 - If composite, the simple attributes together form the primary key
- iii. Any remaining key attributes are kept as secondary unique keys (these will be useful for physical tuning w.r.t. indexing analysis)

Step 1 Result

EMPLOYEE

Fname	Minit	Lname	<u>Ssn</u>	Bdate	Address	Sex	Salary
-------	-------	-------	------------	-------	---------	-----	--------

DEPARTMENT

Dname	<u>Dnumber</u>
-------	----------------

PROJECT

Pname	<u>Pnumber</u>	Plocation
-------	----------------	-----------

Step 2: Weak Entity Types

- i. For each weak entity type, create a corresponding relation that includes all the simple attributes
- ii. Add as a foreign key all of the primary key attribute(s) in the entity corresponding to the owner entity type
- iii. The primary key is the combination of all the primary key attributes from the owner and the partial key of the weak entity, if any

Step 2 Result

EMPLOYEE

Fname	Minit	Lname	<u>Ssn</u>	Bdate	Address	Sex	Salary
-------	-------	-------	------------	-------	---------	-----	--------

DEPARTMENT

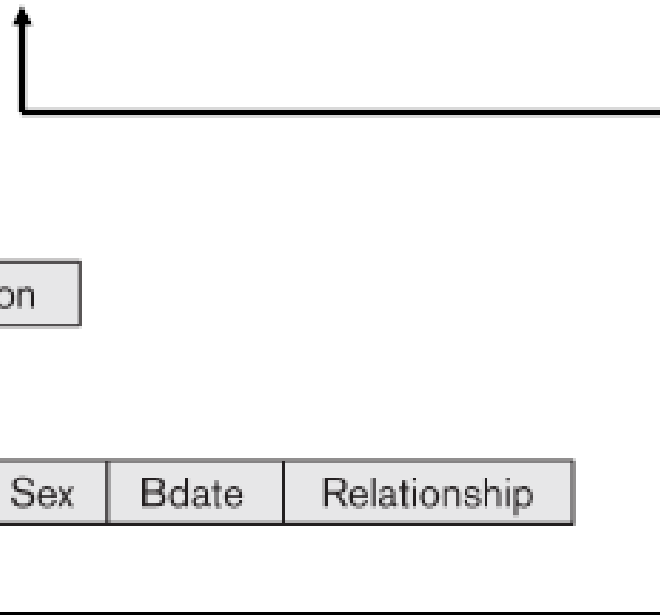
Dname	<u>Dnumber</u>
-------	----------------

PROJECT

Pname	<u>Pnumber</u>	Plocation
-------	----------------	-----------

DEPENDENT

<u>Essn</u>	<u>Dependent_name</u>	Sex	Bdate	Relationship
-------------	-----------------------	-----	-------	--------------



Step 3: Mapping Binary 1-to-1

Three approaches

- **Foreign Key**

- Usually appropriate

- Merged Relation

- Possible when both participations are total

- Relationship Relation

- Not discussed

Step 3: Mapping Binary 1-to-1

Foreign Key

- i. Choose one relation as S , the other T
 - Better if S has total participation (reduces number of NULL values)
- ii. Add to S all the simple attributes of the relationship
- iii. Add as a foreign key in S the primary key attributes of T

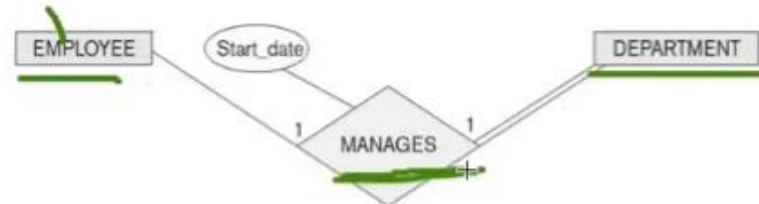
ER-to-Relational Mapping Algorithm

Step 3: Mapping of Binary 1:1 Relation Types

⑤, T

DEPARTMENT

Dname	<u>Dnumber</u>
-------	----------------



DEPARTMENT

Dname	<u>Dnumber</u>	Mgr_ssn	Mgr_start_date
-------	----------------	---------	----------------

Step 3 Result

EMPLOYEE

Fname	Minit	Lname	<u>Ssn</u>	Bdate	Address	Sex	Salary
-------	-------	-------	------------	-------	---------	-----	--------

DEPARTMENT

Dname	<u>Dnumber</u>	Mgr_ssn	Mgr_start_date
-------	----------------	---------	----------------

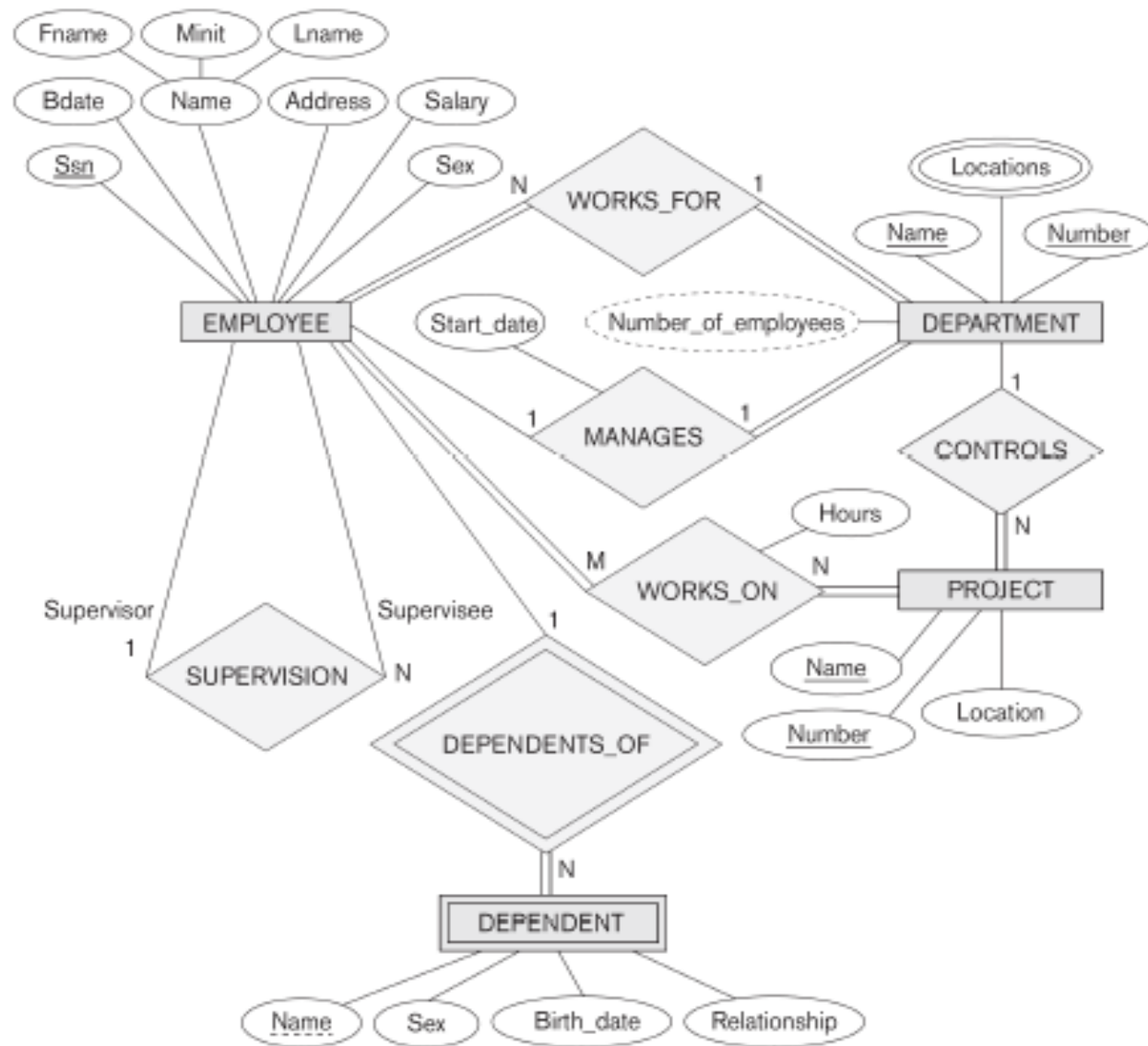


Step 4: Binary 1-to-N

- i. Choose the S relation as the type at the N-side of the relationship, other is T
- ii. Add as a foreign key to S all of the primary key attribute(s) of T

Another approach: create a relationship relation

Example ERD



ER-to-Relational Mapping Algorithm

Step 4: Mapping of Binary 1:N Relationship Types.

EMPLOYEE

Fname	Minit	Lname	<u>Ssn</u>	Bdate	Address	Sex	Salary
-------	-------	-------	------------	-------	---------	-----	--------

EMPLOYEE

Fname	Minit	Lname	<u>Ssn</u>	Bdate	Address	Sex	Salary	Super_ssn	<u>Dno</u>
-------	-------	-------	------------	-------	---------	-----	--------	-----------	------------



PROJECT

Pname	<u>Pnumber</u>	Plocation
-------	----------------	-----------

PROJECT

Pname	<u>Pnumber</u>	Plocation	Dnum
-------	----------------	-----------	------

Step 4 Result

EMPLOYEE

Fname	Minit	Lname	<u>Ssn</u>	Bdate	Address	Sex	Salary	Super_ssn	Dno
-------	-------	-------	------------	-------	---------	-----	--------	-----------	-----

DEPARTMENT

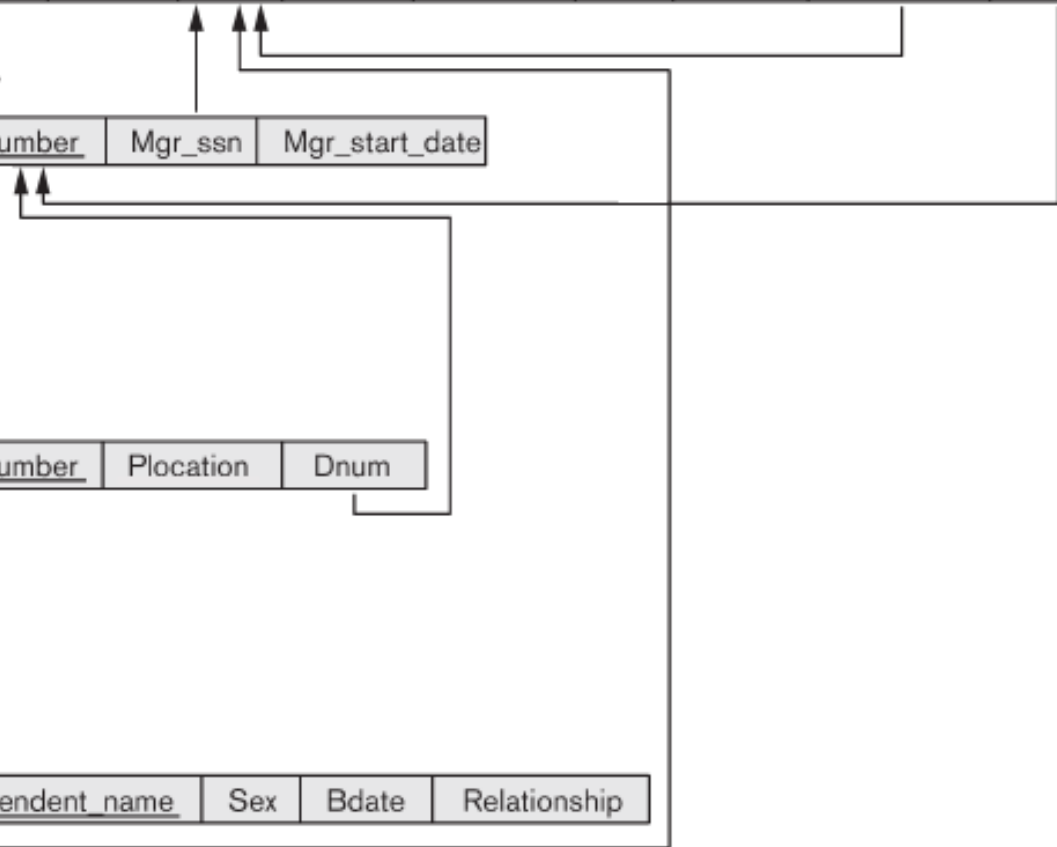
Dname	<u>Dnumber</u>	Mgr_ssn	Mgr_start_date
-------	----------------	---------	----------------

PROJECT

Pname	<u>Pnumber</u>	Plocation	Dnum
-------	----------------	-----------	------

DEPENDENT

<u>Essn</u>	<u>Dependent_name</u>	Sex	Bdate	Relationship
-------------	-----------------------	-----	-------	--------------

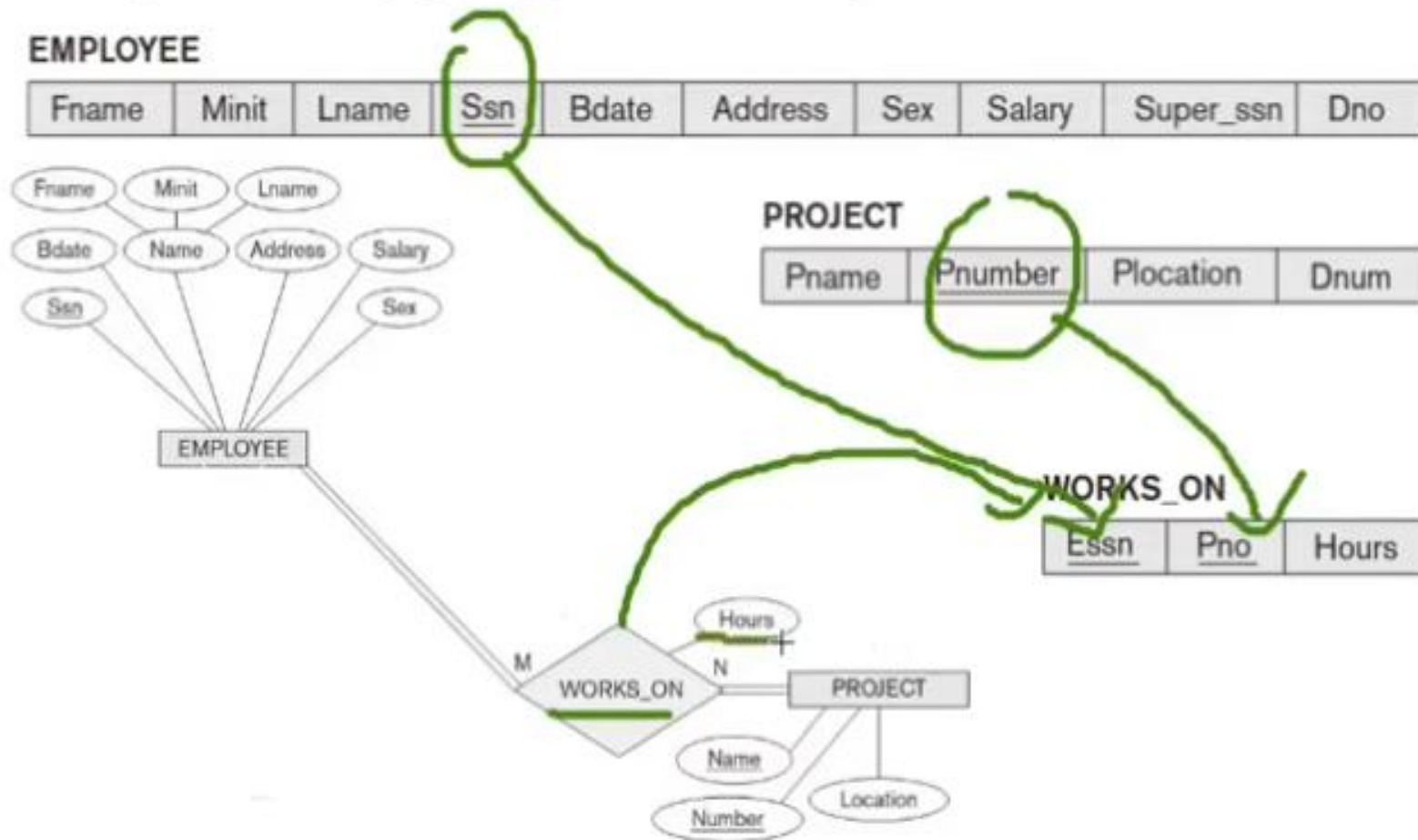


Step 5: Binary M-to-N

- i. Create a new relation S (termed: *relationship relation*)
 - In some ERD dialects, actually drawn in
- ii. Add as foreign keys the primary keys of both relations; their combination forms the primary key of S
- iii. Add any simple attributes of the M:N relationship to S

ER-to-Relational Mapping Algorithm

Step 5: Mapping of Binary M:N Relationship Types



Step 5 Result

EMPLOYEE

Fname	Minit	Lname	<u>Ssn</u>	Bdate	Address	Sex	Salary	Super_ssn	Dno
-------	-------	-------	------------	-------	---------	-----	--------	-----------	-----

DEPARTMENT

Dname	<u>Dnumber</u>	Mgr_ssn	Mgr_start_date
-------	----------------	---------	----------------

PROJECT

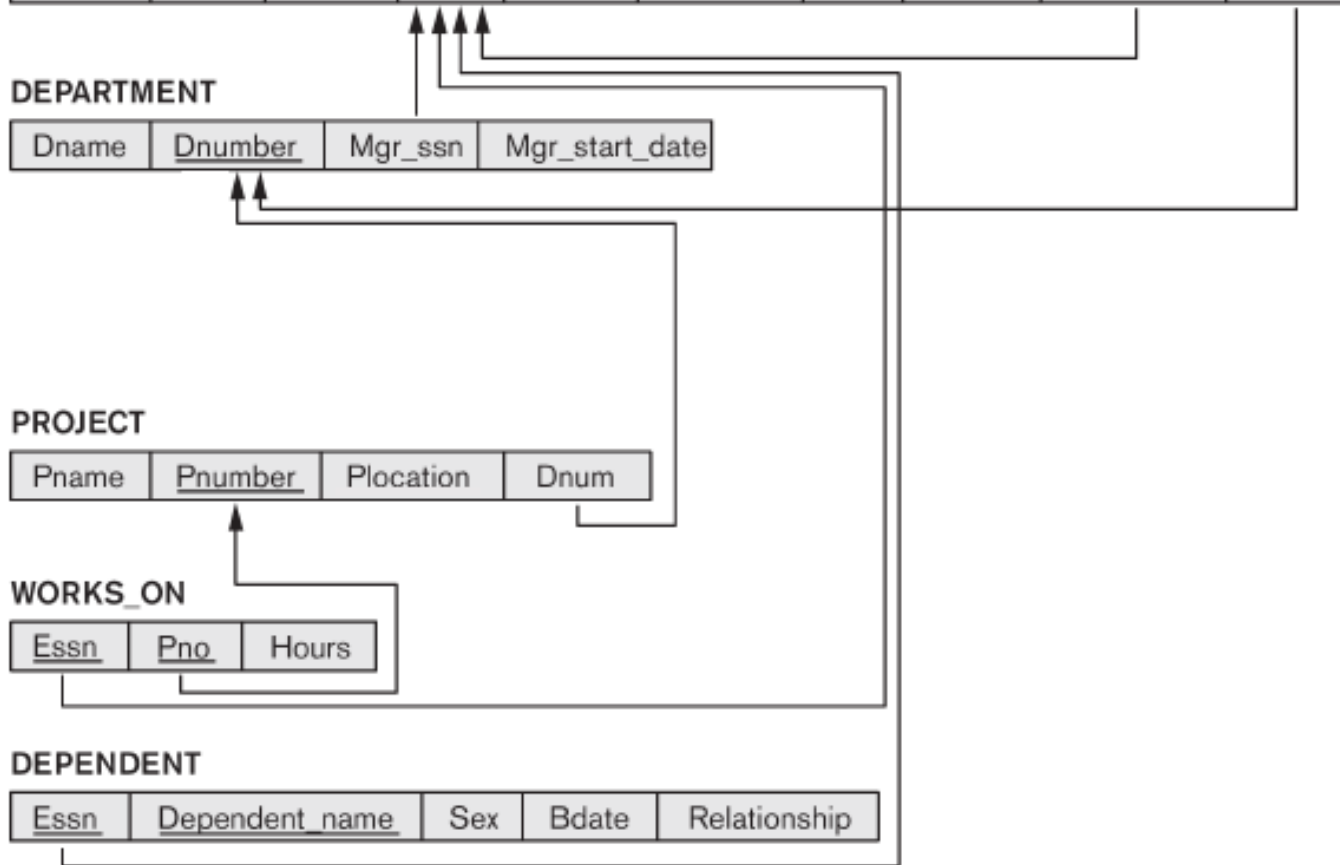
Pname	<u>Pnumber</u>	Plocation	Dnum
-------	----------------	-----------	------

WORKS_ON

<u>Essn</u>	<u>Pno</u>	Hours
-------------	------------	-------

DEPENDENT

<u>Essn</u>	<u>Dependent_name</u>	Sex	Bdate	Relationship
-------------	-----------------------	-----	-------	--------------



Step 6: Multivalued Attributes

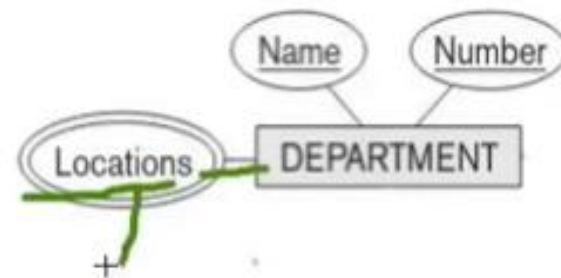
- i. Create a new relation S
- ii. Add as foreign keys the primary keys of the corresponding relation
- iii. Add the attribute to S (if composite, the simple attributes); the combination of all attributes in S forms the primary key

ER-to-Relational Mapping Algorithm

Step 6: Mapping of Multi valued attributes.

DEPT_LOCATIONS

<u>Dnumber</u>	<u>Dlocation</u>
----------------	------------------



Step 6 Result

EMPLOYEE

Fname	Minit	Lname	<u>Ssn</u>	Bdate	Address	Sex	Salary	Super_ssn	Dno
-------	-------	-------	------------	-------	---------	-----	--------	-----------	-----

DEPARTMENT

Dname	<u>Dnumber</u>	Mgr_ssn	Mgr_start_date
-------	----------------	---------	----------------

DEPT_LOCATIONS

<u>Dnumber</u>	<u>Dlocation</u>
----------------	------------------

PROJECT

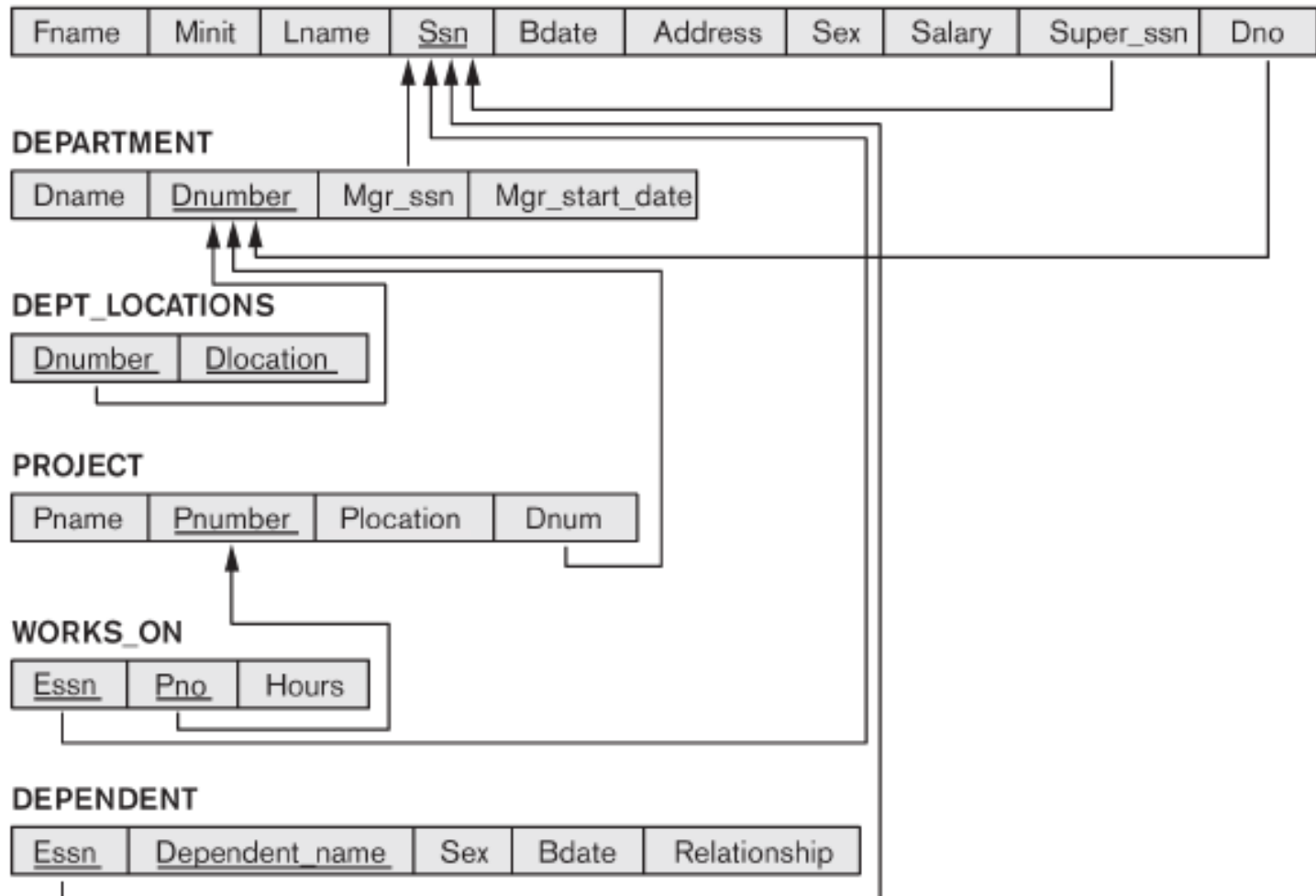
Pname	<u>Pnumber</u>	Plocation	Dnum
-------	----------------	-----------	------

WORKS_ON

<u>Essn</u>	<u>Pno</u>	Hours
-------------	------------	-------

DEPENDENT

<u>Essn</u>	<u>Dependent_name</u>	Sex	Bdate	Relationship
-------------	-----------------------	-----	-------	--------------



ER-to-Relational Mapping Algorithm

• **Step 7: Mapping of N-ary Relationship Types.**

- For each n-ary relationship type R, where $n > 2$, create a new relationship S to represent R.
- Include as foreign key attributes in S the primary keys of the relations that represent the participating entity types.
- Also include any simple attributes of the n-ary relationship type (or simple components of composite attributes) as attributes of S.

• **Example:** The relationship type SUPPY in the ER on the next slide.

- This can be mapped to the relation SUPPLY shown in the relation schema, whose primary key is the combination of the three foreign keys {SNAME, PARTNO, PROJNAME}



(a)



SUPPLIER

<u>SNAME</u>	...
--------------	-----

PROJECT

<u>PROJNAME</u>	...
-----------------	-----

PART

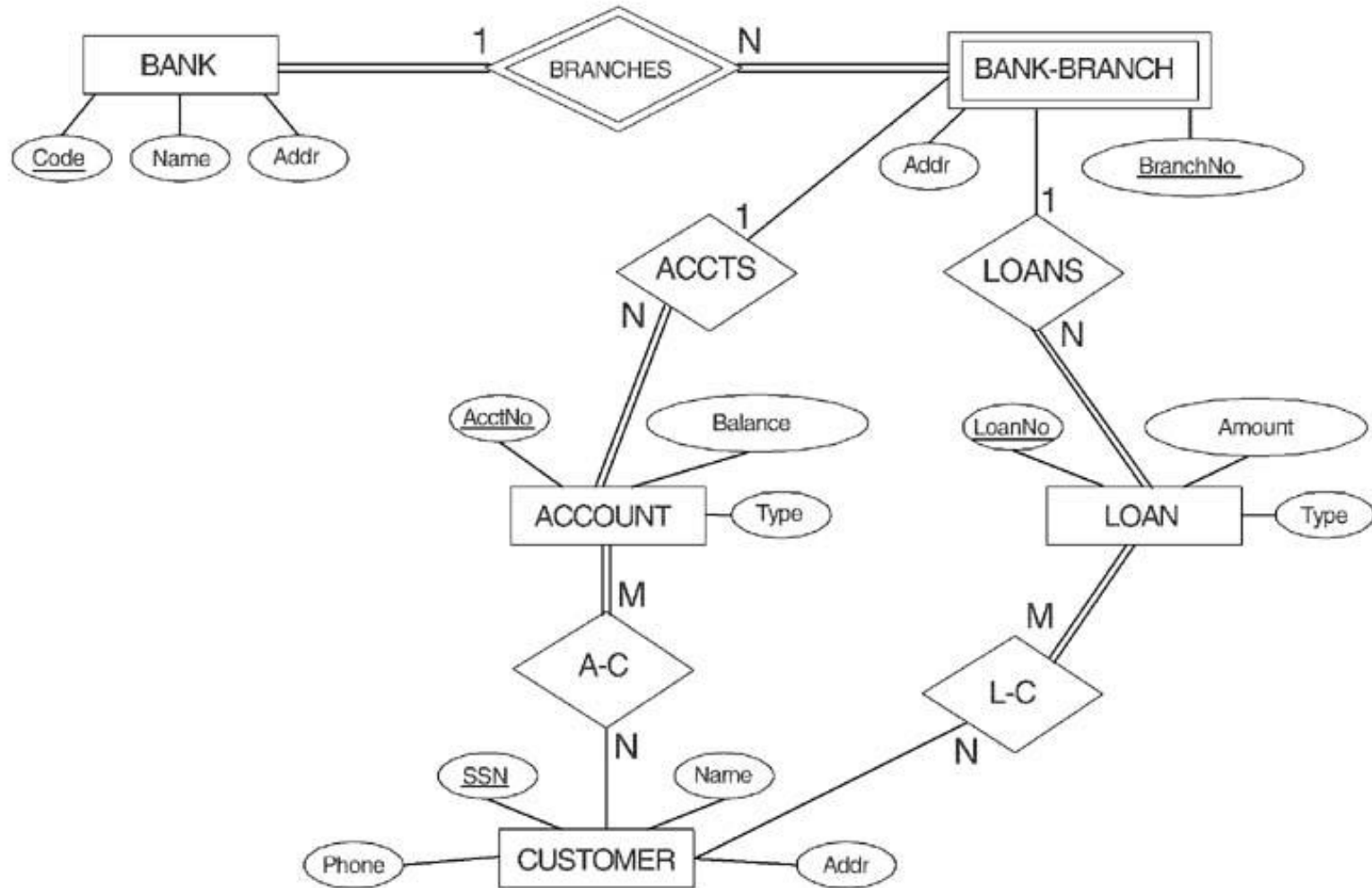
<u>PARTNO</u>	...
---------------	-----

SUPPLY

<u>SNAME</u>	<u>PROJNAME</u>	<u>PARTNO</u>	QUANTITY
--------------	-----------------	---------------	----------



ER DIAGRAM FOR A BANK DATABASE



Data Definition, Constraints, and Schema Changes

Used to CREATE, DROP, and ALTER the descriptions of the tables (relations) of a database